Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 19

Inferencia y generación

Decodificación, caché KV, decodificación especulativa, cuantización y servicio a escala

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Un modelo de lenguaje entrenado es una distribución condicionada $P(x_t \mid x_{<t})$; para producir texto hay que decidir qué token se emite en cada paso y hacerlo deprisa, para miles de usuarios a la vez y con una memoria limitada. Este módulo estudia ambas cuestiones. La primera es estadística: qué regla de decodificación se aplica a la distribución del modelo (búsqueda voraz, búsqueda en haz, muestreo con temperatura, *top-k* o núcleo) y qué propiedades tiene cada una. La segunda es de sistemas: por qué la generación token a token está limitada por el ancho de banda de memoria y no por el cómputo, cómo la caché de claves y valores evita recalcular el pasado, cómo la decodificación especulativa emite varios tokens por pasada sin alterar la distribución, cómo la cuantización reduce los bytes que hay que leer y cómo se organiza un servidor que atiende muchas peticiones simultáneas. Sobre la inferencia de Claude, Gemini y Grok solo se usa información pública: sus API documentan parámetros de muestreo, pero los formatos numéricos, el hardware y los planificadores con los que se sirven no son públicos.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. formular la decodificación como búsqueda o como muestreo, demostrar las propiedades de la temperatura, de *top-k* y del muestreo de núcleo, y aplicar la búsqueda en haz con normalización por longitud;
2. distinguir las fases de *prefill* y de decodificación, demostrar que la caché KV no altera la salida, calcular su tamaño y deducir con el modelo *roofline* la cota de velocidad de la generación y el efecto del tamaño del lote;
3. enunciar el algoritmo de decodificación especulativa, demostrar que preserva exactamente la distribución del modelo objetivo y calcular el número esperado de tokens por pasada y la aceleración;
4. definir la cuantización afín y la simétrica, acotar su error, relacionar bits y relación señal-ruido y analizar el efecto de la granularidad y de los valores atípicos;
5. explicar y justificar los métodos de cuantización de LLM (LLM.int8(), SmoothQuant, GPTQ, AWQ y QLoRA) y la destilación de conocimiento;
6. razonar sobre latencia y rendimiento de un servidor, y analizar el *batching* continuo y la gestión paginada de la caché KV.

Requisitos previos

Del módulo 1 se usan la regla de la cadena, la función softmax con temperatura y los métodos de muestreo elementales; del módulo 5, esperanza, varianza y distribuciones discretas; del módulo 6, entropía y entropía cruzada; del módulo 11, los formatos de coma flotante; de los

módulos 13 a 15, la atención causal, el bloque Transformer y el mini-GPT; del módulo 16, el coste de unas $2N$ operaciones por token en la pasada hacia delante; del módulo 17, la intensidad aritmética y el modelo *roofline*, y del módulo 18, la atención con consultas agrupadas (GQA).

Organización

Tema	Contenido	Fuentes principales
1	Decodificación: búsqueda y muestreo	[13], [22], [11], [5]
2	Coste de la inferencia y caché KV	[9], [17], [14], [18]
3	Decodificación especulativa	[15], [1], [9]
4	Fundamentos de la cuantización	[7], [12], [8], [20]
5	Cuantización de LLM y destilación	[3], [20], [6], [16], [4], [10]
6	Servicio a escala: latencia, rendimiento y memoria	[9], [21], [14], [17]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: ⋆ comprensión, ⋆⋆ cálculo y ⋆⋆⋆ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La organización sigue la clase 10 («Inference») del curso CS336 de Stanford, «Language Modeling from Scratch» [9], que recorre la intensidad aritmética de la inferencia, la caché KV, la cuantización, la decodificación especulativa, el *batching* continuo y PagedAttention, y la sección 6 de su tarea 1 [2], dedicada a la generación. La decodificación sigue el §7.6 y el §13.4 de Jurafsky y Martin [13] y el §10.8 de Zhang *et al.* [22]. La cuantización sigue las clases 5 y 6 («Quantization»), 9 («Knowledge Distillation») y 13 («Efficient LLM Deployment») del curso 6.5940 del MIT [8], el estudio clásico de Gray y Neuhoff [7] y el esquema de aritmética entera de Jacob *et al.* [12]. Los métodos se toman de sus artículos originales: Holtzman *et al.* [11] y Fan *et al.* [5] para el muestreo; Pope *et al.* [17] y Shazeer [18] para el coste de la inferencia; Leviathan *et al.* [15] y Chen *et al.* [1] para la decodificación especulativa; Dettmers *et al.* [3, 4], Xiao *et al.* [20], Frantar *et al.* [6] y Lin *et al.* [16] para la cuantización; Hinton *et al.* [10] para la destilación, y Yu *et al.* [21] y Kwon *et al.* [14] para el servicio. Las arquitecturas que se usan como ejemplo (Llama 2, Llama 3, OPT) son públicas; sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública.

Notación

Símbolo	Significado
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
$x_{<t} = (x_1, \ldots, x_{t-1})$	prefijo (contexto) en el paso t
$z \in \mathbb{R}^K$, $\tau > 0$	vector de logits y temperatura (la letra T se reserva para longitudes)
$p_\tau = \mathrm{softmax}(z/\tau)$	distribución con temperatura
k, p, m	parámetros de *top-k*, del muestreo de núcleo y anchura del haz
N, n	parámetros del modelo y longitud del prompt
L, n_{kv}, d_{h}	capas, cabezas de clave y valor, y dimensión de cada cabeza
s	bytes por número (2 en fp16 y bf16, 1 en int8)
B, B_{mem}, P_{pico}	tamaño del lote, ancho de banda de memoria y rendimiento máximo
p, q	en el tema 3, distribución del modelo objetivo y del borrador
γ, β, α, c	tokens propuestos, tasa de aceptación, su valor esperado y coste relativo del borrador
$\mathrm{TV}(p, q)$	distancia de variación total, $\frac{1}{2}\sum_x \lvert p(x) - q(x)\rvert$
b, Δ, Z	bits, paso (escala) y punto cero de un cuantizador
$H = 2XX^\top$	matriz hessiana del error cuadrático de una capa (tema 5)
TTFT, TPOT	tiempo hasta el primer token y tiempo por token de salida

Índice

Presentación del módulo **ii**

1 Decodificación: búsqueda y muestreo **1**
1.1 El problema de la decodificación . . . 1
1.2 Búsqueda voraz y búsqueda en haz . . . 2
1.3 Muestreo con temperatura . . . 3
1.4 Truncamiento: *top-k* y muestreo de núcleo . . . 4
Ejercicios del tema 1 . . . 5

2 El coste de la inferencia: *prefill*, decodificación y caché KV **7**
2.1 Las dos fases de la generación . . . 7
2.2 La caché KV . . . 7
2.3 Intensidad aritmética de la inferencia . . . 9
2.4 El lote: latencia frente a rendimiento . . . 10
Ejercicios del tema 2 . . . 11

3 Decodificación especulativa **13**
3.1 El algoritmo . . . 13
3.2 Muestreo por rechazo modificado . . . 14
3.3 Corrección para secuencias . . . 15
3.4 Rendimiento . . . 15
Ejercicios del tema 3 . . . 16

4 Fundamentos de la cuantización **18**
4.1 Cuantizadores uniformes . . . 18
4.2 Error de cuantización . . . 18
4.3 Granularidad y valores atípicos . . . 20
4.4 Producto de matrices en aritmética entera . . . 20
Ejercicios del tema 4 . . . 21

5 Cuantización de LLM y destilación **23**
5.1 Rasgos atípicos y LLM.int8() . . . 23
5.2 SmoothQuant: migrar la dificultad a los pesos . . . 24
5.3 GPTQ: cuantización con información de segundo orden . . . 24
5.4 AWQ: proteger los canales importantes . . . 25

5.5 QLoRA: cuantizar a 4 bits para ajustar . . . 26
5.6 Destilación de conocimiento . . . 26
Ejercicios del tema 5 . . . 27

6 Servicio a escala: latencia, rendimiento y memoria 29
6.1 Métricas de servicio . . . 29
6.2 *Batching* estático y continuo . . . 29
6.3 Memoria de la caché KV y PagedAttention . . . 30
Ejercicios del tema 6 . . . 32

A Soluciones de los ejercicios 33
A.1 Tema 1 . . . 33
A.2 Tema 2 . . . 36
A.3 Tema 3 . . . 39
A.4 Tema 4 . . . 41
A.5 Tema 5 . . . 45
A.6 Tema 6 . . . 48

Bibliografía 52

Tema 1

Decodificación: búsqueda y muestreo

Un modelo de lenguaje autorregresivo proporciona, en cada paso, una distribución $P(\cdot \mid x_{<t})$ sobre el vocabulario; la *decodificación* es la regla que convierte esa sucesión de distribuciones en un texto concreto. Este tema presenta las dos familias de reglas: las de *maximización* (búsqueda voraz y búsqueda en haz), que intentan encontrar la secuencia más probable, y las de *muestreo* (temperatura, *top-k* y núcleo), que extraen una secuencia aleatoria de una versión modificada de la distribución. Se siguen el §7.6 y el §13.4 de Jurafsky y Martin [13], el §10.8 de Zhang *et al.* [22], la sección 3 de Holtzman *et al.* [11], el §5.4 de Fan *et al.* [5] y la sección 6 de la tarea 1 de CS336 [2].

1.1 El problema de la decodificación

Sea P un modelo de lenguaje sobre $\mathcal{V}$ con símbolo de fin $\langle . \rangle$ (módulo 1). Por la regla de la cadena, la probabilidad de una continuación $y = (y_1, \ldots, y_T)$ de un prompt x es

$$P(y \mid x) = \prod_{t=1}^{T} P(y_t \mid x, y_{<t}), \qquad \ln P(y \mid x) = \sum_{t=1}^{T} \ln P(y_t \mid x, y_{<t}).$$

Definición 1.1 (Regla de decodificación). Una *regla de decodificación* es un procedimiento que, a partir del prompt x y del acceso a las distribuciones condicionadas $P(\cdot \mid x, y_{<t})$, produce una continuación y, de forma determinista o aleatoria. La *decodificación de máxima probabilidad* (MAP) busca

$$y^* = \arg\max_{y} \; P(y \mid x),$$

donde el máximo se toma sobre las secuencias terminadas en $\langle . \rangle$.

Proposición 1.2. *Con un vocabulario de K tokens, el número de continuaciones de longitud exactamente T es K^T. En consecuencia, la búsqueda exhaustiva del máximo entre las secuencias de longitud T requiere evaluar K^T secuencias.*

Demostración. Cada una de las T posiciones admite K tokens de forma independiente; por el principio del producto hay K^T secuencias (módulo 1). ∎

Con $K = 5 \cdot 10^4$ y $T = 20$, $K^T \approx 9{,}5 \cdot 10^{93}$: la decodificación MAP exacta es inabordable y se aproxima con búsquedas locales [22, §10.8.2].

1.2 Búsqueda voraz y búsqueda en haz

Definición 1.3 (Búsqueda voraz). La *búsqueda voraz* (*greedy*) elige en cada paso el token más probable, $y_t = \arg\max_{v \in \mathcal{V}} P(v \mid x, y_{<t})$, hasta emitir $\langle . \rangle$ o alcanzar una longitud máxima [13, §7.6.1].

Proposición 1.4. *La búsqueda voraz no encuentra, en general, la secuencia de máxima probabilidad.*

Demostración. Basta un contraejemplo. Sea un modelo con primer token A, B o C, con probabilidades 0,5, 0,4 y 0,1, y segundo token x, y o z seguido de $\langle . \rangle$. Tras A el segundo token tiene distribución (0,4; 0,3; 0,3) y tras B, (0,9; 0,05; 0,05). La búsqueda voraz elige A y luego x, con probabilidad $0{,}5 \cdot 0{,}4 = 0{,}20$, mientras que la secuencia Bx tiene probabilidad $0{,}4 \cdot 0{,}9 = 0{,}36 > 0{,}20$. ■

Definición 1.5 (Búsqueda en haz). Sea $m \geq 1$ la *anchura del haz*. La *búsqueda en haz* mantiene un conjunto $\mathcal{H}_t$ de, a lo sumo, m prefijos de longitud t. En el paso $t{+}1$ forma todas las extensiones (h, v) con $h \in \mathcal{H}_t$ y $v \in \mathcal{V}$, las puntúa con $\ln P(h, v \mid x) = \ln P(h \mid x) + \ln P(v \mid x, h)$ y conserva las m de mayor puntuación. Las extensiones que terminan en $\langle . \rangle$ se retiran del haz como candidatas finales; al acabar se devuelve la candidata de mayor puntuación [13, §13.4], [22, §10.8.3].

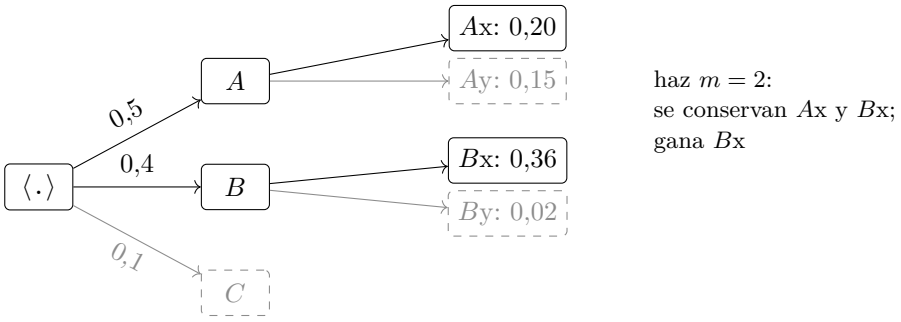


Figura 1.1: Búsqueda en haz con $m = 2$ en el ejemplo de la proposición 1.4. Los nodos discontinuos se podan (no se dibujan las extensiones con z). La búsqueda voraz se queda con Ax (0,20); el haz encuentra Bx (0,36).

Proposición 1.6 (Coste y casos extremos). *Para una salida de longitud T y vocabulario de tamaño K, la búsqueda en haz de anchura m evalúa a lo sumo mKT extensiones. Con $m = 1$ coincide con la búsqueda voraz, y con $m \geq K^{T-1}$ no poda ningún prefijo y coincide con la búsqueda exhaustiva entre las secuencias de longitud T.*

Demostración. En cada uno de los T pasos se extienden a lo sumo m prefijos con K tokens. Si $m = 1$, conservar la mejor extensión del único prefijo es elegir el arg max del paso, que es la regla voraz. En el paso t hay a lo sumo $K^t \leq K^{T-1}$ prefijos posibles para $t \leq T - 1$; si $m \geq K^{T-1}$, ninguno se descarta y en el último paso se puntúan las K^T secuencias. ■

Observación 1.7 (Sesgo hacia las secuencias cortas). Cada término $\ln P(y_t \mid x, y_{<t})$ es menor o igual que cero, de modo que la puntuación $\ln P(y \mid x)$ no aumenta al alargar la secuencia. La búsqueda en haz tiende entonces a preferir candidatas cortas. La corrección habitual divide

por una potencia de la longitud,

$$\operatorname{punt}(y) = \frac{1}{L^{a}} \ln P(y \mid x), \qquad L = |y|,\ a \in [0,1],$$

con $a = 0{,}75$ en el ejemplo de [22, §10.8.3]; con $a = 1$ se obtiene la log-probabilidad media por token de [13, §13.4].

Observación 1.8 (Degeneración). La búsqueda en haz es la regla habitual en traducción automática, donde la salida está muy determinada por la entrada. En la generación abierta produce textos repetitivos: Holtzman *et al.* muestran que el texto humano tiene una probabilidad por token claramente menor que la del texto obtenido por maximización y que las repeticiones se refuerzan a sí mismas, porque cada repetición aumenta la probabilidad de la siguiente [11, §4.3]. Por eso los asistentes conversacionales muestrean.

1.3 Muestreo con temperatura

Definición 1.9 (Softmax con temperatura). Sea $z \in \mathbb{R}^{K}$ el vector de logits y sea $\tau > 0$. La *distribución con temperatura* τ es

$$p_{\tau}(i) = \frac{e^{z_i/\tau}}{\sum_{j=1}^{K} e^{z_j/\tau}}, \qquad i = 1, \dots, K.$$

El *muestreo con temperatura* extrae cada token de p_{τ} [13, §7.6.3], [11, §3.3]. Con $\tau = 1$ se recupera la distribución del modelo.

Proposición 1.10 (Temperatura como potencia). *Sea $p = p_1$ la distribución del modelo. Entonces $p_{\tau}(i) \propto p(i)^{1/\tau}$. En particular, elevar las probabilidades a una potencia $a > 0$ y renormalizar equivale a muestrear con temperatura $\tau = 1/a$.*

Demostración. Como $p(i) = e^{z_i} / \sum_j e^{z_j}$, se tiene $p(i)^{1/\tau} = e^{z_i/\tau} \big(\sum_j e^{z_j}\big)^{-1/\tau}$. El segundo factor no depende de i y desaparece al normalizar, luego $p(i)^{1/\tau} / \sum_j p(j)^{1/\tau} = p_{\tau}(i)$. ∎

Proposición 1.11 (Límites de la temperatura). *Sea $M = \{i : z_i = \max_j z_j\}$ el conjunto de logits máximos. Entonces*

$$\lim_{\tau \to 0^{+}} p_{\tau}(i) = \begin{cases} 1/|M|, & i \in M, \\ 0, & i \notin M, \end{cases} \qquad \lim_{\tau \to \infty} p_{\tau}(i) = \frac{1}{K}.$$

Si el máximo es único, el límite $\tau \to 0^{+}$ es la búsqueda voraz.

Demostración. Sea $z^{*} = \max_j z_j$. Multiplicando numerador y denominador por $e^{-z^{*}/\tau}$,

$$p_{\tau}(i) = \frac{e^{(z_i - z^{*})/\tau}}{\sum_j e^{(z_j - z^{*})/\tau}}.$$

Cuando $\tau \to 0^{+}$, cada exponente $(z_j - z^{*})/\tau$ vale 0 si $j \in M$ y tiende a $-\infty$ si $j \notin M$; el denominador tiende a $|M|$ y el numerador a 1 o a 0. Cuando $\tau \to \infty$, todos los exponentes tienden a 0, el numerador a 1 y el denominador a K. ∎

Ejemplo 1.12. Sean los logits $z = (2, 1, 0, -1)$ de los tokens A, B, C, D. Con $\tau = 1$, $p_1 = (0{,}6439;\ 0{,}2369;\ 0{,}0871;\ 0{,}0321)$. Con $\tau = 0{,}5$ la distribución se afila, $p_{0{,}5} = (0{,}8650;\ 0{,}1171;\ 0{,}0158;\ 0{,}0021)$, y con $\tau = 2$ se aplana, $p_2 = (0{,}4551;\ 0{,}2760;\ 0{,}1674;\ 0{,}1015)$.

La temperatura reescala todos los logits por igual y nunca anula un token: la cola de tokens improbables conserva masa positiva para cualquier $\tau > 0$. Los métodos de truncamiento la eliminan.

1.4 Truncamiento: *top-k* y muestreo de núcleo

Definición 1.13 (Distribución truncada). Sea p una distribución sobre $\mathcal{V}$ y sea $S \subseteq \mathcal{V}$ con $p(S) = \sum_{i \in S} p(i) > 0$. La *distribución truncada* a S es $p_S(i) = p(i)/p(S)$ si $i \in S$ y $p_S(i) = 0$ en otro caso.

Definición 1.14 (*Top-k* y núcleo). Sea $p_{(1)} \geq p_{(2)} \geq \cdots \geq p_{(K)}$ la ordenación decreciente de las probabilidades.

(a) El muestreo *top-k* [5, §5.4] trunca p al conjunto $V^{(k)}$ de los k tokens más probables.

(b) El *muestreo de núcleo* o *top-p* [11, §3.1], con umbral $p \in (0, 1]$, trunca al menor conjunto $V^{(p)}$ tal que $\sum_{i \in V^{(p)}} p(i) \geq p$.

Proposición 1.15 (El núcleo está formado por los más probables). *Sea* $k^* = \min\{k : \sum_{i=1}^{k} p_{(i)} \geq p\}$. *Entonces el tamaño mínimo de un conjunto con masa al menos* p *es* k^**, y el conjunto de los* k^* *tokens más probables lo alcanza. Es decir, el muestreo de núcleo coincide con* top-k *con* $k = k^*$*, donde* k^* *depende de la distribución de cada paso.*

Demostración. Para cualquier conjunto S con $|S| = k$ se cumple $p(S) \leq \sum_{i=1}^{k} p_{(i)}$, porque la suma de k probabilidades cualesquiera no supera la suma de las k mayores. Si $k < k^*$, por la definición de k^*, $p(S) \leq \sum_{i \leq k} p_{(i)} < p$; luego ningún conjunto con menos de k^* elementos alcanza la masa p. El conjunto de los k^* más probables tiene masa $\sum_{i \leq k^*} p_{(i)} \geq p$, de modo que el mínimo es k^*. ∎

Proposición 1.16 (Distorsión del truncamiento). *Para cualquier* S *con* $p(S) > 0$*, la distancia de variación total entre la distribución truncada y la original es la masa eliminada:*

$$\mathrm{TV}(p_S, p) = \tfrac{1}{2} \sum_{i \in \mathcal{V}} |p_S(i) - p(i)| = 1 - p(S).$$

Demostración. Para $i \in S$, $p_S(i) - p(i) = p(i)\big(1/p(S) - 1\big) \geq 0$, y sumando sobre S se obtiene $1 - p(S)$. Para $i \notin S$, $|p_S(i) - p(i)| = p(i)$, y sumando sobre el complementario se obtiene $1 - p(S)$. La mitad de la suma total es $1 - p(S)$. ∎

Así, el muestreo de núcleo con umbral p garantiza $\mathrm{TV}(p_S, p) \leq 1 - p$ en cada paso, mientras que *top-k* con k fijo no controla la masa eliminada: si el modelo duda entre muchas opciones razonables, k fijo descarta masa grande; si está seguro, conserva tokens de cola [11, §3.2].

Ejemplo 1.17. Con la distribución p_1 del ejemplo 1.12, las probabilidades acumuladas son 0,6439, 0,8808, 0,9679 y 1. *Top-k* con $k = 2$ conserva A y B con (0,7311; 0,2689) y elimina una masa 0,1192. El núcleo con $p = 0{,}9$ necesita tres tokens ($0{,}8808 < 0{,}9 \leq 0{,}9679$) y da (0,6652; 0,2447; 0,0900); la masa eliminada, y por tanto la distancia de variación total, es 0,0321.

Observación 1.18 (Combinación y uso práctico). Las reglas se componen en el orden de la figura 1.2: los logits se dividen por τ, se aplica la softmax, se trunca con *top-k* y luego con el núcleo, se renormaliza y se muestrea [13, §7.6.4]. Las API públicas de Anthropic, Google

y xAI exponen la temperatura y el umbral del núcleo como parámetros de la petición; los valores que usan por defecto sus aplicaciones de chat no son públicos.

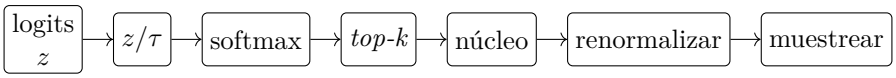


Figura 1.2: Cadena habitual de muestreo. Con $\tau \to 0$ o con $k = 1$ se recupera la búsqueda voraz.

Lecturas recomendadas para este tema

- Jurafsky y Martin [13], §7.6 «Decoding» (voraz, muestreo, temperatura, *top-k* y *top-p*) y §13.4 «Decoding in MT: Beam Search».
- Zhang *et al.* [22], §10.8 «Beam Search».
- Holtzman *et al.* [11], secciones 3 («Language Model Decoding») y 4.3.
- Fan *et al.* [5], §5.4 «Generation».
- CS336 [2], tarea 1, sección 6 «Generating text».

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Compare *top-k* con $k = 50$ y el núcleo con $p = 0{,}9$ en dos situaciones: (a) el modelo está muy seguro, con un token de probabilidad 0,95; (b) el modelo duda entre 200 opciones de probabilidad 0,005 cada una. ¿Cuántos tokens conserva cada método y qué masa elimina?

Ejercicio 1.2 ⋆⋆ Sean los logits $z = (1, 0, -1)$ de los tokens A, B, C. Calcule p_1 y $p_{0{,}5}$, la distribución *top-k* con $k = 2$ (sobre p_1) y los tokens que conserva el núcleo con $p = 0{,}9$. Compruebe que la distribución *top-2* solo depende de la diferencia entre los dos primeros logits.

Ejercicio 1.3 ⋆⋆ Un modelo da los logits $u = (1{,}2;\ 0{,}9;\ 0{,}1;\ -0{,}5)$ para cuatro palabras. (a) Calcule la softmax con $\tau = 1$, $\tau = 0{,}2$ y $\tau = 2$, y compruebe que cada distribución suma 1. (b) Describa qué ocurre cuando $\tau \to 0$ y cuando $\tau \to \infty$, y justifíquelo con la proposición 1.11.

Fuente: adaptado de Jurafsky y Martin [13], cap. 7, ejercicio 7.5 (logits de la figura 7.17).

Ejercicio 1.4 ⋆⋆ Un modelo da logits $v = (1;\ 0{,}5;\ 0;\ -1)$. (a) Calcule la softmax con $\tau = 1$ y con $\tau = 0{,}5$. (b) Aplique el núcleo con $p = 0{,}8$ en ambos casos y dé la distribución renormalizada. (c) ¿Qué ocurre cuando $\tau \to 0$?

Fuente: adaptado de CS336 [2], tarea 1, sección 6 «Generating text», problema `decoding` (temperatura y muestreo de núcleo).

Ejercicio 1.5 ⋆⋆ Vocabulario de $|\mathcal{V}| = 50\,000$ tokens y salida de $T = 20$ tokens. (a) Calcule cuántas extensiones evalúan la búsqueda voraz, la búsqueda en haz con $m = 4$ y la búsqueda exhaustiva. (b) Dos candidatas terminadas: una de 3 tokens con probabilidad 0,12 y otra de

5 tokens con 0,08. Puntúelas con $L^{-a}\ln P$, $a = 0{,}75$. ¿Cuál gana? ¿Y sin normalizar? (c) ¿Es la búsqueda exhaustiva un caso particular de la búsqueda en haz?

Fuente: adaptado de Zhang *et al.* [22], §10.8 «Beam Search», costes de §10.8.1–10.8.3 y ejercicio 1 de §10.8.5.

Ejercicio 1.6 ⋆⋆ Una distribución $P = (0{,}6;\ 0{,}3;\ 0{,}1)$ se transforma en $Q(i) \propto P(i)^{a}$. (a) ¿Qué a corresponde a duplicar la temperatura y cuál a reducirla a la mitad? (b) Calcule Q en ambos casos. (c) ¿Qué ocurre si la temperatura tiende a 0 o a ∞?

Fuente: adaptado de Zhang *et al.* [22], §4.1 «Softmax Regression», ejercicio 7 de §4.1.5.

Ejercicio 1.7 ⋆⋆⋆ Sea $H(\tau) = -\sum_i p_\tau(i)\ln p_\tau(i)$ la entropía de la distribución con temperatura. (a) Demuestre que

$$\frac{\mathrm{d}H}{\mathrm{d}\tau} = \frac{1}{\tau^{3}}\,\mathrm{Var}_{p_\tau}(z) \;\geq\; 0,$$

donde $\mathrm{Var}_{p_\tau}(z) = \sum_i p_\tau(i) z_i^2 - \big(\sum_i p_\tau(i) z_i\big)^2$, de modo que la entropía crece con la temperatura. (b) ¿Cuándo es la derivada nula? (c) Compruebe la fórmula con $z = (1, 0, -1)$ en $\tau = 1$, calculando el miembro derecho.

Tema 2

El coste de la inferencia: *prefill*, decodificación y caché KV

Generar una respuesta tiene dos fases con perfiles de coste opuestos: el *prefill*, que procesa el prompt completo en paralelo, y la *decodificación*, que produce los tokens uno a uno. Este tema demuestra que la caché de claves y valores (caché KV) evita recalcular el pasado sin alterar la salida, calcula su tamaño y aplica el modelo *roofline* del módulo 17 para explicar por qué la decodificación está limitada por el ancho de banda de memoria y cómo el tamaño del lote intercambia latencia por rendimiento. Se siguen la clase 10 de CS336 [9], la sección 2 de Pope *et al.* [17], la sección 3 de Kwon *et al.* [14] y el artículo de Shazeer sobre la atención multiconsulta [18].

2.1 Las dos fases de la generación

Definición 2.1 (*Prefill* y decodificación). Sea un prompt $x_{1:n}$. La fase de *prefill* ejecuta una pasada del Transformer sobre las n posiciones a la vez, como en el entrenamiento, y obtiene la distribución del primer token nuevo. La fase de *decodificación* ejecuta, para cada token nuevo, una pasada sobre una sola posición, la del último token emitido.

Por el módulo 16, una pasada hacia delante cuesta unas $2N$ operaciones por token y posición, donde N es el número de parámetros (sin contar la atención sobre el contexto, que se trata en el proposición 2.9).

Ejemplo 2.2 (Coste del *prefill*). Con $N = 7 \cdot 10^9$ y un prompt de $n = 1000$ tokens, el *prefill* cuesta $2Nn = 1{,}4 \cdot 10^{13}$ FLOP. En un acelerador hipotético de 300 TFLOP/s utilizado al 50 %, tarda $1{,}4 \cdot 10^{13}/(1{,}5 \cdot 10^{14}) \approx 0{,}093$ s. Este tiempo es el núcleo del tiempo hasta el primer token (tema 6).

2.2 La caché KV

En un Transformer causal (módulos 13 y 14), sean $h_i^{(\ell)}$ el estado de la posición i a la salida de la capa ℓ, con $h_i^{(0)}$ la incrustación del token x_i y su posición. Cada capa calcula, para cada posición, una consulta, una clave y un valor por cabeza, $q_i = h_i^{(\ell-1)} W_Q$, $k_i = h_i^{(\ell-1)} W_K$, $v_i = h_i^{(\ell-1)} W_V$, y la salida de la atención en t es

$$a_t = \mathrm{softmax}\Big(\frac{q_t\,[k_1; \dots; k_t]^\top}{\sqrt{d_\mathrm{h}}}\Big)\,[v_1; \dots; v_t],$$

seguida de las conexiones residuales, la normalización y el perceptrón multicapa, que actúan posición a posición.

Lema 2.3 (Causalidad de los estados). *Para toda capa ℓ y toda posición i, el estado $h_i^{(\ell)}$ depende solo de $x_{1:i}$.*

Demostración. Por inducción sobre ℓ. Para $\ell = 0$, $h_i^{(0)}$ depende solo de x_i y de i. Supóngase cierto para $\ell - 1$. La consulta, la clave y el valor de la posición j en la capa ℓ son funciones de $h_j^{(\ell-1)}$, luego dependen solo de $x_{1:j}$. Por la máscara causal, la salida de la atención en i usa q_i y los pares (k_j, v_j) con $j \le i$, que dependen solo de $x_{1:i}$. Las operaciones posteriores actúan sobre la posición i por separado, de modo que $h_i^{(\ell)}$ depende solo de $x_{1:i}$. ■

Teorema 2.4 (Corrección de la caché KV). *Sea un procedimiento que, en cada paso de decodificación, calcula q, k y v únicamente para la posición nueva t en cada capa, añade (k_t, v_t) a una caché que contiene los pares (k_j, v_j), $j < t$, calculados en pasos anteriores, y atiende sobre toda la caché. Entonces sus salidas coinciden exactamente (en aritmética exacta) con las de una pasada completa sobre $x_{1:t}$.*

Demostración. Por el lema 2.3, los pares (k_j, v_j) de cada capa para $j < t$ dependen solo de $x_{1:j}$; al añadir x_t no cambian, y la pasada completa sobre $x_{1:t}$ recalcularía exactamente los valores guardados. La salida en la posición t de la pasada completa usa q_t y los pares con $j \le t$, que son los mismos que usa el procedimiento con caché. Por inducción sobre las capas, los estados $h_t^{(\ell)}$ coinciden en ambos cálculos y, por tanto, también los logits. ■

En coma flotante pueden aparecer diferencias de redondeo, porque el orden de las operaciones cambia; la distribución que define el modelo es la misma.

Proposición 2.5 (Pasadas de posición con y sin caché). *Sea un prompt de n tokens del que se generan T tokens. Sin caché, el número total de posiciones procesadas por las capas no atencionales es $Tn + T(T-1)/2$; con caché es $n + T - 1$.*

Demostración. Sin caché, la pasada que produce el token j-ésimo procesa las $n + j - 1$ posiciones existentes, y $\sum_{j=1}^{T}(n + j - 1) = Tn + \sum_{j=0}^{T-1} j = Tn + T(T-1)/2$. Con caché, la primera pasada es el *prefill* de n posiciones y las $T - 1$ restantes procesan una posición cada una. ■

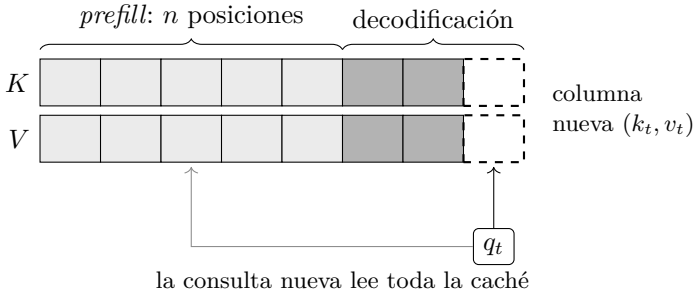


Figura 2.1: Caché KV de una capa. El *prefill* rellena las n primeras columnas; cada paso de decodificación calcula solo la consulta, la clave y el valor de la posición nueva, añade una columna y atiende sobre todas.

Proposición 2.6 (Tamaño de la caché KV). *Sea un modelo con L capas, n_{kv} cabezas de clave y valor de dimensión d_{h}, y números de s bytes. La caché ocupa*

$$c_{\text{kv}} = 2\, L\, n_{\text{kv}}\, d_{\text{h}}\, s \quad \textit{bytes por token,}$$

y una secuencia de S tokens ocupa $S\,c_{\mathrm{kv}}$ bytes.

Demostración. Por cada token y cada capa se guarda un vector clave y un vector valor (factor 2) por cada una de las n_{kv} cabezas, cada uno con d_{h} números de s bytes. ■

Ejemplo 2.7 (Arquitecturas públicas). Llama 2 7B tiene $L = 32$, 32 cabezas con atención multicabeza clásica ($n_{\mathrm{kv}} = 32$) y $d_{\mathrm{h}} = 128$. En fp16, $c_{\mathrm{kv}} = 2 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 524\,288$ bytes $= 512$ KiB por token; una secuencia de 4096 tokens ocupa 2 GiB, y 32 secuencias, 64 GiB, más que los unos 13 GB de pesos. Llama 3 8B usa GQA con $n_{\mathrm{kv}} = 8$ y los mismos L y d_{h}: $c_{\mathrm{kv}} = 131\,072$ bytes $= 128$ KiB, cuatro veces menos. Del mismo modo, OPT-13B ($L = 40$, 40 cabezas de dimensión 128) necesita $2 \cdot 40 \cdot 5120 \cdot 2 = 819\,200$ bytes, unos 800 KiB por token [14, §3].

Observación 2.8 (Atención multiconsulta y agrupada). La caché es proporcional a n_{kv}. La atención multiconsulta (MQA) [18] usa $n_{\mathrm{kv}} = 1$ y la agrupada (GQA, módulo 18) un valor intermedio; respecto a la atención multicabeza con n_h cabezas, la caché se divide por n_h/n_{kv}. Pope *et al.* muestran que la atención multiconsulta permite contextos hasta 32 veces más largos con la misma memoria en PaLM 540B [17].

2.3 Intensidad aritmética de la inferencia

Se recuerda del módulo 17 que la intensidad aritmética de una operación es $I = F/Q$ (FLOP por byte transferido) y que, en un dispositivo con rendimiento P_{pico} y ancho de banda B_{mem}, la operación está limitada por la memoria si $I < I^* = P_{\mathrm{pico}}/B_{\mathrm{mem}}$.

Proposición 2.9 (Intensidad de las capas). *Sea un lote de B secuencias en bf16 ($s = 2$) del que se procesan T posiciones nuevas por secuencia, con S posiciones ya presentes en la caché.*

(a) En un perceptrón con tres matrices de $D \times F$ (como el SwiGLU), $F_{\mathrm{op}} = 6BTDF$ y $Q = 4BTD + 4BTF + 6DF$. Si $BT \ll D, F$, la intensidad es $I \approx BT$.

(b) En la atención, con dimensión total D de consultas, claves y valores, $F_{\mathrm{op}} = 4BSTD$ y $Q = 4BSD + 4BTD$, de modo que $I = ST/(S + T)$, independiente de B. En el prefill *($T = S$), $I = S/2$; en la decodificación ($T = 1$), $I = S/(S + 1) < 1$.*

Demostración. (a) Cada uno de los tres productos de $(BT \times D)$ por $(D \times F)$ cuesta $2BTDF$ operaciones, en total $6BTDF$. Se leen la entrada (BTD números) y los tres pesos ($3DF$), se escriben dos activaciones intermedias ($2BTF$; como en [9], su relectura no se cuenta) y se escribe la salida (BTD); con dos bytes por número, $Q = 2(2BTD + 2BTF + 3DF) = 4BTD + 4BTF + 6DF$. Si $BT \ll D, F$, el término $6DF$ domina y $I \approx 6BTDF/(6DF) = BT$.

(b) El producto de las consultas $(B \times T \times D)$ por las claves $(B \times S \times D)$ cuesta $2BSTD$ y el de los pesos de atención por los valores, otras $2BSTD$. Se leen Q, K y V ($BTD + 2BSD$ números) y se escribe la salida (BTD), es decir, $Q = 2(2BTD + 2BSD)$ bytes. Así, $I = 4BSTD/(4BSD + 4BTD) = ST/(S + T)$. Sustituyendo $T = S$ y $T = 1$ se obtienen los dos casos. ■

La diferencia entre ambas capas es estructural: en el perceptrón todas las secuencias del lote comparten los mismos pesos, que se leen una vez; en la atención cada secuencia tiene su propia caché, de modo que agrupar secuencias no reutiliza datos [9].

Teorema 2.10 (Cota de memoria de la decodificación). *Sea un modelo de N parámetros de s bytes y sea un paso de decodificación con un lote de B secuencias de S tokens de contexto cada una. Si los pesos y la caché se leen al menos una vez por paso, el tiempo del paso cumple*

$$t_{\text{paso}} \;\geq\; \frac{sN + B\,S\,c_{\text{kv}}}{B_{\text{mem}}}.$$

En particular, con $B = 1$ y caché despreciable, la velocidad de generación cumple tokens/s $\leq B_{\text{mem}}/(sN)$.

Demostración. Por la cota del modelo *roofline* (módulo 17), el tiempo de un cálculo que transfiere Q bytes es al menos Q/B_{mem}. En un paso se leen los sN bytes de pesos y las B cachés de $S\,c_{\text{kv}}$ bytes. Con $B = 1$, cada paso produce un token, luego la velocidad es $1/t_{\text{paso}} \leq B_{\text{mem}}/(sN)$. ∎

Ejemplo 2.11 (Cota de velocidad). Un modelo de $7 \cdot 10^9$ parámetros en fp16 ocupa $1{,}4 \cdot 10^{10}$ bytes. Con $B_{\text{mem}} = 2$ TB/s, $t_{\text{paso}} \geq 7$ ms y la velocidad no supera $2 \cdot 10^{12}/(1{,}4 \cdot 10^{10}) \approx 143$ tokens/s, independientemente del rendimiento de cómputo. Si el dispositivo ofrece 300 TFLOP/s, $I^* = 150$ FLOP/byte; la decodificación con lote 1 tiene $I \approx 1$ y usa menos del 1 % de su capacidad de cálculo.

2.4 El lote: latencia frente a rendimiento

Proposición 2.12 (Compromiso del lote). *En las condiciones del teorema 2.10, supóngase que el tiempo de un paso es exactamente $t(B) = t_0 + Bt_1$, con $t_0 = sN/B_{\text{mem}}$ y $t_1 = S\,c_{\text{kv}}/B_{\text{mem}}$. Entonces la velocidad por usuario $1/t(B)$ decrece con B, el rendimiento total $R(B) = B/t(B)$ crece con B y*

$$\lim_{B\to\infty} R(B) = \frac{1}{t_1} = \frac{B_{\text{mem}}}{S\,c_{\text{kv}}}.$$

Demostración. $t(B)$ es creciente en B, luego $1/t(B)$ es decreciente. Además $R(B) = B/(t_0 + Bt_1) = 1/(t_0/B + t_1)$, y t_0/B decrece con B, de modo que R crece y tiende a $1/t_1$. ∎

El lote amortiza la lectura de los pesos, pero no la de la caché. La cuadro 2.1 lo ilustra con Llama 2 7B en fp16 (14 GB de pesos), secuencias de 2000 tokens (1,05 GB de caché cada una) y $B_{\text{mem}} = 2$ TB/s; el límite del rendimiento es $B_{\text{mem}}/(S\,c_{\text{kv}}) \approx 1907$ tokens/s. Además, la memoria total limita B: con 64 secuencias la caché ya ocupa 67 GB.

Lote B	Caché total (GB)	Tiempo por paso (ms)	Tokens/s totales	Tokens/s por usuario
1	1,05	7,52	133	133
8	8,39	11,19	715	89,3
32	33,55	23,78	1346	42,1
64	67,11	40,55	1578	24,7

Cuadro 2.1: Cota de memoria del teorema 2.10 para Llama 2 7B en fp16 con contexto de 2000 tokens y $B_{\text{mem}} = 2$ TB/s.

Observación 2.13 (Cuándo deja de mandar la memoria). Respecto a los pesos, la intensidad de un paso con lote B es $\approx B$ FLOP/byte en bf16 (proposición 2.9), de modo que haría falta $B \approx I^*$ para quedar limitado por el cómputo; la caché, con intensidad menor que 1,

lo impide antes. Pope *et al.* cuantifican este compromiso en PaLM 540B: el *prefill* de lotes grandes alcanza un 76 % de utilización de FLOP, mientras que la decodificación con lote pequeño logra una latencia de 29 ms por token con pesos en int8 [17]. Todo lo que reduzca bytes acelera la decodificación: menos cabezas de clave y valor, cuantización (temas 4 y 5) y más tokens por lectura de pesos (tema 3).

Lecturas recomendadas para este tema

- CS336 [9], clase 10 «Inference»: intensidad aritmética de la inferencia, latencia y rendimiento, reducción de la caché KV.
- Pope *et al.* [17], sección 2 «Inference Cost Tradeoffs» y §3.3 sobre la partición de la atención multiconsulta.
- Kwon *et al.* [14], sección 3 «Memory Challenges in LLM Serving».
- Shazeer [18], artículo original de la atención multiconsulta.

Ejercicios del tema 2

Ejercicio 2.1 ⋆ ¿Por qué el *prefill* está limitado por el cómputo y la decodificación por la memoria, si ejecutan el mismo modelo? Razónelo con la intensidad aritmética respecto a los pesos.

Ejercicio 2.2 ⋆ Justifique si son verdaderas o falsas: (a) la caché KV cambia los tokens que genera el modelo; (b) duplicar el tamaño del lote duplica el rendimiento total; (c) con GQA de 32 a 8 cabezas de clave y valor caben cuatro veces más secuencias en la memoria reservada a la caché.

Ejercicio 2.3 ⋆⋆ Un modelo tiene $L = 24$ capas y $n_{\text{kv}} = 16$ cabezas de clave y valor de dimensión $d_{\text{h}} = 64$, en fp16. Calcule la caché KV por token, para una secuencia de 2048 tokens y para un lote de 16 secuencias de 2048 tokens.

Ejercicio 2.4 ⋆⋆ Un modelo de $1{,}3 \cdot 10^{10}$ parámetros se sirve en un acelerador de 1 TB/s. Calcule la cota de tokens/s con lote 1 en fp16, en int8 y en int4 (medio byte por peso, sin contar escalas).

Ejercicio 2.5 ⋆⋆ Se modela Llama 2 13B en bf16 con $N = 1{,}3015 \cdot 10^{10}$ parámetros, $L = 40$, 40 cabezas de clave y valor de dimensión 128 y contexto $S = 1024$, en una GPU H100 con $B_{\text{mem}} = 3{,}35$ TB/s y 80 GB de memoria. Suponga que en cada paso se leen todos los pesos y todas las cachés. (a) Calcule la caché por secuencia. (b) Para $B = 1$, 64 y 256, calcule la memoria total, la latencia por paso y el rendimiento. (c) ¿Qué lotes caben en la GPU? Comente el resultado.

Fuente: adaptado de CS336 [9], clase 10 «Inference», apartado *throughput and latency* (Llama 2 13B en una H100).

Ejercicio 2.6 ⋆⋆⋆ Demuestre que, en un paso de decodificación con lote B y pesos en fp16, la intensidad aritmética respecto a los pesos es B FLOP/byte. Estime con qué lote el

acelerador del ejemplo 2.11 dejaría de estar limitado por los pesos y explique por qué la caché KV impide llegar ahí.

Ejercicio 2.7 ⋆⋆⋆ Considere solo los productos de la atención de una capa con dimensión D y genere T tokens a partir de un prompt vacío. (a) Demuestre que sin caché el coste total de la atención es $\sum_{t=1}^{T} 4Dt^2 = \frac{2}{3}DT(T+1)(2T+1)$ y, por tanto, $\Theta(T^3)$. (b) Demuestre que con caché es $\sum_{t=1}^{T} 4Dt = 2DT(T+1)$, es decir, $\Theta(T^2)$. (c) Calcule el cociente para $T = 1000$.

Tema 3

Decodificación especulativa

El tema 2 mostró que un paso de decodificación está limitado por la lectura de los pesos: procesar unas pocas posiciones más en la misma pasada apenas cuesta tiempo. La *decodificación especulativa* aprovecha esta asimetría entre generar y verificar. Un modelo pequeño (el *borrador*) propone varios tokens y el modelo grande (el *objetivo*) los verifica todos en una sola pasada con una regla de aceptación y rechazo que preserva *exactamente* su distribución. Se siguen el artículo de Leviathan, Kalman y Matias [15] (secciones 2 y 3 y apéndice A.1), el de Chen *et al.* [1], que propuso el mismo algoritmo de forma independiente con el nombre de *speculative sampling*, y la clase 10 de CS336 [9].

3.1 El algoritmo

En todo el tema, $p(\cdot \mid x_{<t})$ es la distribución del modelo objetivo y $q(\cdot \mid x_{<t})$ la del borrador, ambas ya modificadas por la regla de muestreo que se desee (temperatura, núcleo, etc.). Cuando el contexto está fijado se escribe simplemente $p(x)$ y $q(x)$.

Definición 3.1 (Ronda de decodificación especulativa). Sea $\gamma \geq 1$ un entero y sea $x_{<t}$ el prefijo actual. Una *ronda* consiste en:

1. El borrador genera $\tilde{x}_1, \ldots, \tilde{x}_\gamma$ de forma autorregresiva, con $\tilde{x}_i \sim q(\cdot \mid x_{<t}, \tilde{x}_{<i})$, y guarda esas probabilidades.
2. El modelo objetivo procesa en una sola pasada el prefijo y los γ tokens propuestos, y obtiene $p(\cdot \mid x_{<t}, \tilde{x}_{<i})$ para $i = 1, \ldots, \gamma + 1$.
3. Para $i = 1, \ldots, \gamma$, en orden, se acepta $\tilde{x}_i$ con probabilidad $\min\big(1,\, p(\tilde{x}_i)/q(\tilde{x}_i)\big)$, con ambas distribuciones condicionadas al mismo prefijo.
4. En el primer rechazo, en la posición i, se emite un token de la *distribución residual*

$$r(x) = \frac{\max\big(0,\, p(x) - q(x)\big)}{\sum_y \max\big(0,\, p(y) - q(y)\big)}$$

   y la ronda termina.
5. Si se aceptan los γ tokens, se emite además un token de $p(\cdot \mid x_{<t}, \tilde{x}_{1:\gamma})$, que ya se calculó en el paso 2.

Cada ronda emite entre 1 y $\gamma + 1$ tokens con una sola pasada del modelo objetivo. Esa pasada procesa $\gamma + 1$ posiciones, como un pequeño *prefill*: está limitada por la lectura de los pesos y cuesta casi lo mismo que un paso de decodificación (teorema 2.10).

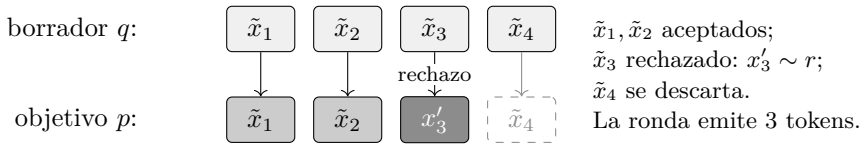


Figura 3.1: Una ronda con $\gamma = 4$. El borrador propone cuatro tokens (arriba); el objetivo los evalúa en una pasada y acepta los dos primeros. En el primer rechazo emite un token de la distribución residual y la ronda termina.

3.2 Muestreo por rechazo modificado

Se analiza primero un solo token. Se propone $\tilde{x} \sim q$, que se acepta con probabilidad $\min(1, p(\tilde{x})/q(\tilde{x}))$; si se rechaza, se emite $x' \sim r$.

Definición 3.2 (Tasa de aceptación). La *tasa de aceptación* es la probabilidad de aceptar la propuesta [15, definición 3.1]:

$$\beta = \sum_x q(x) \min\Big(1, \frac{p(x)}{q(x)}\Big) = \sum_x \min\big(p(x), q(x)\big).$$

Lema 3.3 (Normalización del residuo). $\sum_x \max\big(0, p(x) - q(x)\big) = 1 - \beta$. *En consecuencia, r está bien definida si $\beta < 1$, y $\beta = 1$ si y solo si $p = q$.*

Demostración. Para cada x, $\max(0, p - q) = p - \min(p, q)$: si $p \le q$ ambos miembros valen 0, y si $p > q$ ambos valen $p - q$. Sumando, $\sum_x \max(0, p - q) = 1 - \beta$. Si $\beta = 1$, la suma de los términos no negativos $\max(0, p - q)$ es nula, luego $p \le q$ en todo punto; como ambas suman 1, $p = q$. El recíproco es inmediato. ∎

Teorema 3.4 (Preservación de la distribución). *El token emitido por el muestreo por rechazo modificado tiene distribución exactamente p.*

Demostración. Si $\beta = 1$, $p = q$ por el lema 3.3, nunca se rechaza y el token emitido sigue $q = p$. Supóngase $\beta < 1$. El token emitido es x si se propone x y se acepta, o si se rechaza la propuesta (sea cual sea) y el residuo produce x. El primer suceso tiene probabilidad $q(x) \min(1, p(x)/q(x)) = \min(p(x), q(x))$. El rechazo tiene probabilidad $1 - \beta$ y es independiente del muestreo del residuo, de modo que

$$P(\text{emitir } x) = \min\big(p(x), q(x)\big) + (1-\beta)\,\frac{\max\big(0, p(x) - q(x)\big)}{1 - \beta} = \min(p, q) + \max(0, p - q) = p(x),$$

donde la última igualdad es la del lema 3.3, $\max(0, p - q) = p - \min(p, q)$. ∎

Proposición 3.5 (Aceptación y variación total). $1 - \beta = \mathrm{TV}(p, q)$, *de modo que la tasa de rechazo es la distancia de variación total entre el objetivo y el borrador [15, lema 3.3 y teorema 3.5].*

La demostración se pide en el ejercicio 3.5. Un borrador es bueno exactamente en la medida en que su distribución está próxima, en variación total, a la del objetivo; por eso se suele obtener por destilación del objetivo [9] (tema 5).

Ejemplo 3.6. Sea $\mathcal{V} = \{A, B, C\}$ con $p = (0{,}5;\ 0{,}3;\ 0{,}2)$ y $q = (0{,}2;\ 0{,}6;\ 0{,}2)$. Las probabilidades de aceptación son $\min(1; 2{,}5) = 1$ para A, $\min(1; 0{,}5) = 0{,}5$ para B y 1 para C; $\beta = 0{,}2 + 0{,}3 + 0{,}2 = 0{,}7$. El residuo es $\max(0, p - q) = (0{,}3;\ 0;\ 0)$, normalizado $(1, 0, 0)$. Entonces $P(A) = 0{,}2 + 0{,}3 \cdot 1 = 0{,}5$, $P(B) = 0{,}6 \cdot 0{,}5 = 0{,}3$ y $P(C) = 0{,}2$: exactamente p. El borrador sobrestima B; el rechazo elimina el exceso y el residuo lo devuelve a A, que estaba infrarrepresentado. Una simulación de $2 \cdot 10^5$ rondas da frecuencias (0,5003; 0,3004; 0,1993).

3.3 Corrección para secuencias

Teorema 3.7 (Preservación en secuencias). *Sea un procedimiento que ejecuta rondas sucesivas (definición 3.1) a partir de un prompt, cada una sobre el prefijo que dejó la anterior. Para todo T, los T primeros tokens emitidos tienen la misma distribución conjunta que T pasos de muestreo autorregresivo de p.*

Demostración. Por la regla de la cadena (módulo 1), basta probar que, condicionado a cualquier historia de tokens ya emitidos $y_{<j}$ (y a cualquier estado interno de la ronda en curso compatible con ella), el token j-ésimo emitido sigue $p(\cdot \mid x, y_{<j})$. Dentro de una ronda hay tres casos. (i) El token procede de una propuesta $\tilde{x}_i$ aceptada o del residuo tras rechazarla: como los tokens anteriores de la ronda se aceptaron, el prefijo de $\tilde{x}_i$ es exactamente $y_{<j}$, la propuesta se extrajo de $q(\cdot \mid x, y_{<j})$ y la decisión de aceptarla usa p y q condicionadas a ese prefijo; por el teorema 3.4, el token emitido sigue $p(\cdot \mid x, y_{<j})$. (ii) Es el token adicional tras aceptar las γ propuestas: se extrae directamente de $p(\cdot \mid x, y_{<j})$. (iii) Es el primer token de una ronda nueva: la ronda empieza con el prefijo $y_{<j}$ y se aplica (i) o (ii). En los tres casos la distribución condicionada es la del objetivo, y la conjunta coincide con la del muestreo autorregresivo. ■

Corolario 3.8 (Versión voraz). *Si p y q son distribuciones degeneradas en sus respectivos* $\arg\max$ *(búsqueda voraz), la regla de aceptación acepta $\tilde{x}_i$ si y solo si coincide con el* $\arg\max$ *del objetivo, y en caso contrario emite ese* $\arg\max$*. La salida es idéntica a la búsqueda voraz con el objetivo.*

Demostración. Si $\tilde{x}_i$ es el $\arg\max$ de p, $p(\tilde{x}_i)/q(\tilde{x}_i) = 1/1 = 1$ y se acepta; si no, $p(\tilde{x}_i) = 0$ y se rechaza con probabilidad 1. El residuo $\max(0, p - q)$ es positivo solo en el $\arg\max$ de p, que es el token emitido. El resultado se sigue del teorema 3.7. ■

3.4 Rendimiento

Definición 3.9 (Aceptación media y coste relativo). Se supone que las tasas de aceptación de las sucesivas propuestas son variables independientes e idénticamente distribuidas, de media $\alpha = \mathbb{E}[\beta]$ [15, §3.1]. Se llama *coste relativo* c al cociente entre el tiempo de una pasada del borrador y el de una pasada del objetivo [15, definición 3.7].

Teorema 3.10 (Tokens esperados por ronda). *Con aceptaciones independientes de probabilidad $\alpha < 1$, el número N_{r} de tokens emitidos en una ronda cumple*

$$\mathbb{E}[N_{\mathrm{r}}] = \sum_{j=0}^{\gamma} \alpha^j = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}.$$

Demostración. Si se aceptan exactamente $a < \gamma$ propuestas, la ronda emite $a+1$ tokens (las aceptadas y la del residuo); si se aceptan las γ, emite $\gamma+1$. Así, $N_{\mathrm{r}} \geq j+1$ si y solo si se aceptan las j primeras propuestas, suceso de probabilidad α^j, para $j = 0, \ldots, \gamma$. Como N_{r} toma valores en $\{1, \ldots, \gamma+1\}$, $\mathbb{E}[N_{\mathrm{r}}] = \sum_{j \geq 1} P(N_{\mathrm{r}} \geq j) = \sum_{j=0}^{\gamma} \alpha^j$, y la suma geométrica da la fórmula. ■

Teorema 3.11 (Factor de mejora). *En las condiciones anteriores, si el tiempo de una ronda es el de γ pasadas del borrador más una del objetivo, el cociente entre el tiempo esperado por token sin y con decodificación especulativa es*

$$f(\gamma) = \frac{1 - \alpha^{\gamma+1}}{(1-\alpha)(\gamma c + 1)}.$$

Demostración. Se toma como unidad el tiempo de una pasada del objetivo, que sin especulación produce un token. Una ronda cuesta $\gamma c + 1$ unidades y emite en media $\mathbb{E}[N_{\mathrm{r}}]$ tokens. Por la ley de los grandes números aplicada a las rondas, el tiempo medio por token es $(\gamma c+1)/\mathbb{E}[N_{\mathrm{r}}]$, y el cociente con el tiempo sin especulación, 1, es $f(\gamma)$ por el teorema 3.10 [15, teorema 3.8]. ■

Corolario 3.12. *Si $\alpha > c$, existe γ con $f(\gamma) > 1$; en concreto, $f(1) = (1+\alpha)/(1+c) > 1$ [15, corolario 3.9].*

Demostración. Con $\gamma = 1$, $f(1) = (1-\alpha^2)/((1-\alpha)(1+c)) = (1+\alpha)/(1+c)$, que es mayor que 1 si y solo si $\alpha > c$. ■

Ejemplo 3.13 (Elección de γ). Con $\alpha = 0{,}7$ y $c = 0{,}1$, para $\gamma = 4$, $\mathbb{E}[N_{\mathrm{r}}] = (1-0{,}7^5)/0{,}3 = 2{,}773$ tokens por ronda, la ronda cuesta 1,4 pasadas del objetivo y $f(4) = 1{,}981$. Los valores para $\gamma = 1, \ldots, 6$ son 1,545; 1,825; 1,949; 1,981; 1,961 y 1,912: el óptimo es $\gamma = 4$. Como γ es entero, se elige numéricamente [15, §3.5].

Observación 3.14 (Resultados publicados). Leviathan *et al.* obtienen aceleraciones de 2 a 3 veces en T5-XXL con salidas idénticas [15], y Chen *et al.*, de 2 a 2,5 veces en Chinchilla 70B con un borrador de 4000 millones de parámetros [1]. La clase 10 de CS336 describe extensiones en las que el borrador es una pequeña red añadida al propio objetivo [9]. Si Claude, Gemini o Grok emplean decodificación especulativa en su servicio no es información pública.

Lecturas recomendadas para este tema

- Leviathan, Kalman y Matias [15], secciones 2 («Speculative Decoding») y 3 («Analysis») y apéndice A.1 («Correctness of Speculative Sampling»).
- Chen *et al.* [1], algoritmo 2 y teorema 1 del material suplementario.
- CS336 [9], clase 10 «Inference», apartado *speculative sampling*.

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Justifique si son verdaderas o falsas: (a) la decodificación especulativa empeora un poco la calidad a cambio de velocidad; (b) si el borrador coincide con el objetivo, se aceptan todas las propuestas; (c) cada ronda emite al menos un token.

Ejercicio 3.2 ⋆⋆ Con $\mathcal{V} = \{A, B\}$, $p = (0{,}6;\ 0{,}4)$ y $q = (0{,}9;\ 0{,}1)$, calcule la probabilidad de aceptación de cada token, β, la distribución residual, y compruebe que el token emitido sigue p.

Ejercicio 3.3 ⋆⋆ Sea $\mathcal{V} = \{A, B\}$ y suponga que el borrador sobrestima A: $q(A) > p(A)$. (a) Demuestre que $q(B) < p(B)$ y que la distribución residual es $(0, 1)$. (b) Calcule $P(\text{emitir } A)$ y $P(\text{emitir } B)$ en función de p y q y compruebe que valen $p(A)$ y $p(B)$. (c) Aplíquelo con $p = (0{,}3;\ 0{,}7)$ y $q = (0{,}8;\ 0{,}2)$.

Fuente: adaptado de CS336 [9], clase 10 «Inference», demostración por ejemplo del muestreo especulativo con dos tokens (notación de p y q intercambiada).

Ejercicio 3.4 ⋆⋆ Con $\alpha = 0{,}5$: (a) calcule el número esperado de tokens por ronda con $\gamma = 3$ y con $\gamma = 10$; (b) con $c = 0{,}1$, calcule el factor de mejora en ambos casos y para $\gamma = 2$. ¿Merece la pena subir γ?

Ejercicio 3.5 ⋆⋆⋆ (a) Demuestre que si se aceptase siempre la propuesta (sin rechazo ni residuo), el token emitido seguiría q y no p. (b) Demuestre la proposición 3.5: $1 - \beta = \frac{1}{2}\sum_x |p(x) - q(x)|$.

Ejercicio 3.6 ⋆⋆⋆ Sean $0 < \alpha < 1$ y $c > 0$ fijos. (a) Demuestre que $\mathbb{E}[N_{\mathrm{r}}]$ es estrictamente creciente en γ y está acotado por $1/(1 - \alpha)$. (b) Demuestre que $f(\gamma) \to 0$ cuando $\gamma \to \infty$ y deduzca que existe un γ óptimo finito. (c) Demuestre que si $\alpha \leq c$, entonces $f(\gamma) \leq 1$ para todo $\gamma \geq 1$.

Tema 4

Fundamentos de la cuantización

Como la decodificación está limitada por la memoria (tema 2), guardar los pesos con menos bits acelera casi en proporción a la reducción de bytes. *Cuantizar* es representar números reales con un conjunto finito de niveles, normalmente enteros de b bits acompañados de una escala. Este tema define los cuantizadores uniformes afín y simétrico, acota su error, deduce la relación entre bits y relación señal-ruido, analiza la granularidad de las escalas y el efecto de los valores atípicos, y muestra cómo se realiza un producto de matrices en aritmética entera. Se siguen las clases 5 y 6 («Quantization») de MIT 6.5940 [8], el estudio de Gray y Neuhoff [7], la sección 2 de Jacob *et al.* [12], la sección 2 de Xiao *et al.* [20] y el §2.1 de Dettmers *et al.* [3].

4.1 Cuantizadores uniformes

En todo el tema, $\operatorname{round}(u)$ denota el redondeo al entero más próximo con los empates resueltos lejos de cero. Es una función impar, $\operatorname{round}(-u) = -\operatorname{round}(u)$, y cumple $|\operatorname{round}(u) - u| \le \frac{1}{2}$ y $\operatorname{round}(u + n) = \operatorname{round}(u) + n$ para todo entero n.

Definición 4.1 (Cuantizador afín). Sean $b \ge 1$ bits, un *paso* o *escala* $\Delta > 0$ y un *punto cero* entero $Z \in \{0, \dots, 2^b - 1\}$. El *cuantizador afín* asigna a $x \in \mathbb{R}$ el entero

$$Q(x) = \operatorname{clip}\big(\operatorname{round}(x/\Delta) + Z,\ 0,\ 2^b - 1\big),$$

y la *descuantización* es $\hat{x} = \Delta\,(Q(x) - Z)$. Para un grupo de valores con rango $[x_{\min}, x_{\max}]$, con $x_{\min} \le 0 \le x_{\max}$, la calibración por mínimo y máximo toma

$$\Delta = \frac{x_{\max} - x_{\min}}{2^b - 1}, \qquad Z = \operatorname{round}\Big(-\frac{x_{\min}}{\Delta}\Big).$$

Definición 4.2 (Cuantizador simétrico). El *cuantizador simétrico* de b bits usa $Z = 0$, los enteros $\{-(2^{b-1} - 1), \dots, 2^{b-1} - 1\}$ y la escala $\Delta = \max_i |x_i| / (2^{b-1} - 1)$, llamada de *máximo absoluto* (*absmax*). En int8 los enteros van de -127 a 127; LLM.int8() escribe la misma regla como multiplicación por $s_x = 127/\|x\|_\infty = 1/\Delta$ [3, §2.1].

El cuantizador simétrico es más sencillo y es el habitual para los pesos, cuya distribución está centrada en cero; el afín aprovecha mejor los niveles cuando el rango es asimétrico, como en las activaciones posteriores a una ReLU [8].

4.2 Error de cuantización

Teorema 4.3 (Cota del error). *Sea un cuantizador afín calibrado por mínimo y máximo (definición 4.1). Para todo $x \in [x_{\min}, x_{\max}]$, el recorte no actúa y*

$$|\hat{x} - x| \le \frac{\Delta}{2}.$$

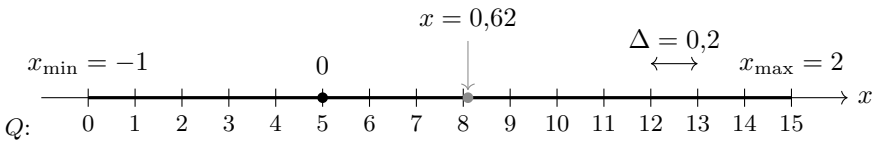


Figura 4.1: Rejilla del cuantizador afín de 4 bits del ejercicio 4.2: los 16 enteros $Q = 0, \ldots, 15$ cubren $[-1, 2]$ con paso $\Delta = 0{,}2$; el real 0 corresponde al punto cero $Z = 5$. El valor $x = 0{,}62$ cae en $x/\Delta + Z = 8{,}1$ y se lleva al entero $Q = 8$, que se reconstruye como $\hat{x} = 0{,}6$.

*Además, $x = 0$ se representa sin error. Para x fuera del rango, el recorte puede actuar y el error no está acotado (*saturación*).*

Demostración. Sea $x \in [x_{\min}, x_{\max}]$. Como round es creciente, $\text{round}(x/\Delta)$ está entre $\text{round}(x_{\min}/\Delta)$ y $\text{round}(x_{\max}/\Delta)$. Por ser round impar, $\text{round}(x_{\min}/\Delta)+Z = \text{round}(x_{\min}/\Delta)+\text{round}(-x_{\min}/\Delta) = 0$. Por la definición de Δ, $x_{\max}/\Delta = x_{\min}/\Delta + (2^b - 1)$, y como $2^b - 1$ es entero, $\text{round}(x_{\max}/\Delta) + Z = 2^b - 1$. Luego $\text{round}(x/\Delta) + Z \in [0, 2^b - 1]$ y el recorte no actúa. Entonces $Q(x) - Z = \text{round}(x/\Delta)$ y

$$|\hat{x} - x| = \Delta \left| \text{round}\Big(\frac{x}{\Delta}\Big) - \frac{x}{\Delta} \right| \leq \frac{\Delta}{2}.$$

Si $x = 0$, $\text{round}(0) = 0$, $Q = Z$ y $\hat{x} = 0$. Si $x > x_{\max}$ es suficientemente grande, $Q(x) = 2^b - 1$ y $\hat{x} = \Delta(2^b - 1 - Z)$ es fijo, de modo que $|\hat{x} - x|$ crece sin límite con x. ■

La representación exacta del cero importa en la práctica: el relleno con ceros y las activaciones nulas no introducen error [12].

Ejemplo 4.4. Sea $x = (-0{,}8;\ -0{,}1;\ 0{,}25;\ 1{,}6)$, con rango de anchura 2,4.

	int8 afín	int4 afín
Δ	$2{,}4/255 = 0{,}009412$	$2{,}4/15 = 0{,}16$
Z	$\text{round}(85{,}0) = 85$	$\text{round}(5{,}0) = 5$
$Q(x)$	$(0;\ 74;\ 112;\ 255)$	$(0;\ 4;\ 7;\ 15)$
$\hat{x}$	$(-0{,}8;\ -0{,}10353;\ 0{,}25412;\ 1{,}6)$	$(-0{,}8;\ -0{,}16;\ 0{,}32;\ 1{,}6)$
error máximo	$0{,}00412 \leq \Delta/2 = 0{,}00471$	$0{,}07 \leq \Delta/2 = 0{,}08$

Proposición 4.5 (Modelo de ruido uniforme). *Si el error $e = \hat{x} - x$ se modela como una variable uniforme en $[-\Delta/2, \Delta/2]$, entonces $\mathbb{E}[e] = 0$ y $\mathbb{E}[e^2] = \Delta^2/12$.*

El modelo es una aproximación clásica, buena cuando el paso es pequeño frente a la escala de variación de la densidad de x [7]; la demostración se pide en el ejercicio 4.6. Una simulación con $\Delta = 0{,}01$ y 10^6 muestras da $\mathbb{E}[e^2] \approx 8{,}345 \cdot 10^{-6}$, frente a $\Delta^2/12 = 8{,}333 \cdot 10^{-6}$.

Corolario 4.6 (Seis decibelios por bit). *Si el rango está fijo, cada bit adicional divide Δ entre 2 y el error cuadrático medio entre 4, de modo que la relación señal-ruido aumenta en $10 \log_{10} 4 \approx 6{,}02$ dB por bit.*

Demostración. Con rango fijo de anchura R, $\Delta_b = R/(2^b - 1)$ y $\Delta_{b+1}/\Delta_b = (2^b - 1)/(2^{b+1} - 1) \approx \frac{1}{2}$; por la proposición 4.5, el error cuadrático se multiplica por $\approx \frac{1}{4}$, y la relación señal-ruido, que es la varianza de la señal dividida por la del error, por ≈ 4. ■

4.3 Granularidad y valores atípicos

Definición 4.7 (Granularidad). Sea una matriz de pesos $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. La cuantización es *por tensor* si usa una sola escala para toda la matriz; *por canal* si usa una escala por cada canal de salida (columna de W), y *por grupos* si divide cada canal en grupos de g pesos consecutivos con una escala cada uno [20, §2]. Para las activaciones se usa además la cuantización *por token*, con una escala por fila.

Proposición 4.8 (Coste de las escalas). *Con pesos de b bits y una escala de b_Δ bits por grupo de g pesos, el almacenamiento medio es $b + b_\Delta/g$ bits por peso.*

Demostración. Cada grupo ocupa $gb + b_\Delta$ bits y contiene g pesos. ∎

Con int4, grupos de $g = 128$ y escalas en fp16, el coste es $4 + 16/128 = 4{,}125$ bits por peso. La cuadro 4.1 traslada estas cifras a la cota de velocidad del teorema 2.10.

Formato	Bytes por peso	Pesos de un modelo de $7 \cdot 10^9$	Cota de tokens/s a 2 TB/s
fp16 / bf16	2	14 GB	143
int8	1	7 GB	286
int4, grupos de 128	0,5156	3,61 GB	554

Cuadro 4.1: Efecto del formato de los pesos en la memoria y en la cota de velocidad con lote 1.

Proposición 4.9 (Efecto de un valor atípico). *En un grupo cuantizado de forma simétrica con b bits, si un valor tiene módulo $M = \max_i |x_i|$, todos los valores con $|x_i| < \Delta/2 = M/\big(2(2^{b-1}-1)\big)$ se cuantizan a cero, y la cota de error del grupo es proporcional a M.*

Demostración. Si $|x_i| < \Delta/2$, entonces $|x_i/\Delta| < \frac{1}{2}$ y $\operatorname{round}(x_i/\Delta) = 0$. La cota del teorema 4.3 es $\Delta/2 = M/\big(2(2^{b-1}-1)\big)$. ∎

Por ejemplo, un grupo int8 con valores del orden de 0,1 y un atípico $M = 8$ tiene $\Delta = 8/127 = 0{,}063$: los valores pequeños quedan representados con solo uno o dos niveles. Los remedios son grupos más pequeños, tratar los atípicos aparte y transformar la matriz antes de cuantizar; se estudian en el tema 5.

4.4 Producto de matrices en aritmética entera

Proposición 4.10 (Producto cuantizado). *Sean $X \in \mathbb{R}^{B \times n}$ y $W \in \mathbb{R}^{n \times d}$ cuantizadas con $X \approx \Delta_X(Q_X - Z_X)$ (afín, por tensor) y $W \approx \Delta_W Q_W$ (simétrica, por tensor). Entonces*

$$XW \approx \Delta_X \Delta_W \big(Q_X Q_W - Z_X \, \mathbf{1}\mathbf{1}^\top Q_W\big),$$

donde $\mathbf{1} \in \mathbb{R}^B$ y $\mathbf{1} \in \mathbb{R}^n$ son vectores de unos. El producto $Q_X Q_W$ se calcula con enteros y acumuladores de 32 bits, y el término $\mathbf{1}^\top Q_W$ (sumas de las columnas de Q_W) puede precalcularse una vez [12, §2].

Demostración. $(Q_X - Z_X \mathbf{1}\mathbf{1}^\top) Q_W = Q_X Q_W - Z_X \mathbf{1}(\mathbf{1}^\top Q_W)$, y basta multiplicar por $\Delta_X \Delta_W$. ∎

Proposición 4.11 (Qué escalas se pueden factorizar). *Sean escalas Δ_i^X por fila de X (por token) y Δ_j^W por columna de W (por canal de salida), con cuantización simétrica. Entonces*

$$(XW)_{ij} \approx \Delta_i^X \, \Delta_j^W \sum_{k=1}^{n} (Q_X)_{ik} (Q_W)_{kj},$$

es decir, la descuantización es un reescalado del producto entero por el producto exterior de las escalas. En cambio, una escala por columna de X (por canal de entrada) depende del índice k que se suma y no se puede sacar del sumatorio.

Demostración. $(XW)_{ij} = \sum_k X_{ik} W_{kj} \approx \sum_k \Delta_i^X (Q_X)_{ik} \, \Delta_j^W (Q_W)_{kj}$, y los factores Δ_i^X y Δ_j^W no dependen de k. Si la escala de X fuese Δ_k^X, cada término del sumatorio llevaría un factor distinto y el producto entero no bastaría. ■

Esta es la cuantización vectorial de LLM.int8() [3, §3.1], y su limitación explica por qué los atípicos de las activaciones, que se concentran en unos pocos *canales*, son difíciles de tratar con núcleos de cálculo enteros [20, §3].

Observación 4.12 (Cuantización durante o después del entrenamiento). La cuantización *posterior al entrenamiento* (PTQ) calibra escalas y puntos cero con unos pocos datos y no modifica el entrenamiento; la cuantización *consciente del entrenamiento* (QAT) simula el redondeo en la pasada hacia delante para que los pesos se adapten, a cambio de un coste de entrenamiento mucho mayor [9, 8]. Los formatos de coma flotante de 8 bits (fp8) se trataron en el módulo 17. Qué formatos usan Anthropic, Google o xAI para servir sus modelos no es público.

Lecturas recomendadas para este tema

- MIT 6.5940 [8], clases 5 y 6 «Quantization» (cuantización lineal, granularidad, PTQ y QAT).
- Gray y Neuhoff [7], en especial el análisis del cuantizador uniforme y la aproximación de alta resolución.
- Jacob *et al.* [12], sección 2 (esquema de cuantización y producto de matrices en aritmética entera).
- Dettmers *et al.* [3], §2.1 «8-bit Data Types and Quantization».

Ejercicios del tema 4

Ejercicio 4.1 ⋆ (a) ¿Por qué se usa habitualmente cuantización simétrica para los pesos y afín para activaciones no negativas? (b) ¿Por qué es deseable que el real 0 se represente exactamente? (c) ¿Qué ventaja tiene una escala por canal frente a una escala por tensor?

Ejercicio 4.2 ⋆⋆ Cuantice $x = (-1;\ 0;\ 0{,}62;\ 2)$ en int4 afín calibrado por mínimo y máximo. Dé Δ, Z, los enteros, los valores reconstruidos y el error máximo, y compruebe el teorema 4.3. ¿Qué ocurriría con $x = 3$ y la misma calibración?

Ejercicio 4.3 ⋆⋆ Cuantice

$$W = \begin{pmatrix} 1{,}2 & -0{,}4 & 0{,}25 \\ -1 & 2{,}6 & 0 \\ 0{,}5 & -1{,}6 & 1{,}9 \end{pmatrix}$$

con cuantización afín de 3 bits (enteros $0, \dots, 7$) por tensor. Dé Δ, Z, la matriz de enteros, la reconstruida y la matriz de errores, y compruebe la cota $\Delta/2$.

Fuente: adaptado de University of Washington, CSE 493G1/599G1 [19], sección 4 «Quantization», ejercicio de práctica de cuantización a 3 bits.

Ejercicio 4.4 ⋆⋆ Sea $v = (0{,}12;\ -0{,}31;\ 0{,}05;\ 0{,}27;\ -0{,}08;\ 0{,}19;\ -0{,}22;\ 6{,}4)$ y cuantización simétrica int4 (enteros $-7, \dots, 7$). (a) Cuantice v por tensor y dé el error máximo. (b) Cuantice por grupos de 4. (c) Guarde el atípico en fp16 y cuantice los otros 7 valores con una sola escala. Compare los errores máximos.

Ejercicio 4.5 ⋆⋆ (a) En un producto int8 por int8 con enteros en $[-127, 127]$ y acumulador int32 (máximo $2^{31} - 1$), ¿cuál es la mayor dimensión interna n que garantiza que no hay desbordamiento? (b) Calcule los bits por peso de int4 con escalas fp16 en grupos de 64 y de 128, y la memoria de un modelo de $7 \cdot 10^9$ parámetros en cada caso.

Ejercicio 4.6 ⋆⋆⋆ (a) Demuestre la proposición 4.5. (b) Sea una señal uniforme en $[-A, A]$ cuantizada con 2^b niveles equiespaciados de paso $\Delta = 2A/2^b$. Demuestre que, con el modelo de ruido uniforme, la relación señal-ruido es exactamente 2^{2b}, es decir, $20\, b \log_{10} 2 \approx 6{,}02\, b$ dB. (c) Calcúlela para $b = 4$ y $b = 8$.

Ejercicio 4.7 ⋆⋆⋆ (a) Demuestre que, para cualquier matriz W y cuantización simétrica, la cota de error $\Delta_j/2$ de la cuantización por canal no supera en ningún canal la cota $\Delta/2$ de la cuantización por tensor. (b) Sean $X = \begin{pmatrix} 1 & 4 \end{pmatrix}$ y $W = \begin{pmatrix} 1 & 1 \end{pmatrix}^{\top}$. Con escalas por columna de X, $\Delta^X = (1, 4)$, se tiene $Q_X = (1, 1)$, y con $\Delta_W = 1$, $Q_W = (1, 1)^{\top}$. Compruebe que $\sum_k \Delta_k^X (Q_X)_k \, \Delta_W (Q_W)_k = XW = 5$, mientras que el producto entero $Q_X Q_W$ multiplicado por Δ_W y por cualquiera de las escalas disponibles de X no da 5, como afirma la proposición 4.11.

Tema 5

Cuantización de LLM y destilación

Los cuantizadores del tema 4 funcionan bien en redes pequeñas, pero en los LLM aparecen dos dificultades: activaciones con unos pocos canales de magnitud muy grande y la pérdida de precisión al bajar a 4 bits o menos. Este tema estudia los métodos que las resuelven: la descomposición en precisión mixta de LLM.int8(), la migración de la dificultad de SmoothQuant, la corrección de segundo orden de GPTQ, el escalado según las activaciones de AWQ y el formato NF4 de QLoRA. Termina con la destilación de conocimiento, que comprime el modelo cambiando su tamaño en lugar de su formato. Se siguen los artículos originales [3, 20, 6, 16, 4, 10], las clases 6 («Quantization, Part II»), 9 («Knowledge Distillation») y 13 («Efficient LLM Deployment») de MIT 6.5940 [8] y la clase 10 de CS336 [9].

5.1 Rasgos atípicos y LLM.int8()

Dettmers *et al.* observan que, a partir de unos $6{,}7 \cdot 10^9$ parámetros, las activaciones de los Transformer presentan *rasgos atípicos*: unas pocas dimensiones ocultas con valores de magnitud mayor o igual que 6, presentes de forma sistemática en casi todas las capas [3, §4]. Por la proposición 4.9, una escala por token se ve dominada por ellos y destruye la precisión del resto.

Definición 5.1 (Descomposición en precisión mixta). Sean $X \in \mathbb{R}^{B\times n}$ y $W \in \mathbb{R}^{n\times d}$ y sea $\alpha_0 > 0$ un umbral. Sea $O \subseteq \{1,\dots,n\}$ el conjunto de dimensiones internas h en las que alguna entrada de la columna X^h supera α_0 en valor absoluto. *LLM.int8()* calcula

$$XW \approx \sum_{h\in O} X^h W_h \;+\; S \cdot \sum_{h\notin O} Q(X^h)\, Q(W_h),$$

donde X^h es la columna h de X, W_h la fila h de W, el primer término se calcula en fp16 y el segundo en int8 con cuantización vectorial (escala por fila de X y por columna de W, proposición 4.11) y reescalado S [3, §3.2].

Con $\alpha_0 = 6$, los atípicos ocupan del orden del 0,1 % de las dimensiones, de modo que el 99,9 % de los productos se hace en 8 bits; en modelos de hasta $13 \cdot 10^9$ parámetros hay como mucho 7 dimensiones atípicas [3, §3.2].

Proposición 5.2 (Exactitud de la descomposición). *La descomposición de la definición 5.1 es exacta salvo por el error de cuantización del segundo término: $XW = \sum_{h\in O} X^h W_h + \sum_{h\notin O} X^h W_h$.*

Demostración. El producto de matrices es la suma de los productos exteriores de columnas de X por filas de W, $XW = \sum_{h=1}^{n} X^h W_h$, y basta separar los índices de O de los demás. ∎

5.2 SmoothQuant: migrar la dificultad a los pesos

Los pesos son fáciles de cuantizar y las activaciones no. SmoothQuant traslada parte de la magnitud de cada canal de entrada de las activaciones a la fila correspondiente de los pesos mediante una transformación exacta [20, §4].

Proposición 5.3 (Equivalencia del suavizado). *Para todo vector $s \in \mathbb{R}^n$ con componentes positivas, $XW = \big(X\,\mathrm{diag}(s)^{-1}\big)\big(\mathrm{diag}(s)W\big)$.*

Demostración. $\mathrm{diag}(s)^{-1}\,\mathrm{diag}(s) = I$ y el producto de matrices es asociativo. ∎

Definición 5.4 (Factores de suavizado). Sea $\alpha \in [0,1]$ la *intensidad de migración*. SmoothQuant toma, para cada canal j,

$$s_j = \frac{\max_i |X_{ij}|^{\alpha}}{\max_k |W_{jk}|^{1-\alpha}},$$

con estadísticas de X obtenidas en un conjunto de calibración [20, ec. 4].

Proposición 5.5 (Equilibrio con $\alpha = \frac{1}{2}$). *Sean $\hat{X} = X\,\mathrm{diag}(s)^{-1}$ y $\hat{W} = \mathrm{diag}(s)W$, y sean $m_j^X = \max_i |X_{ij}|$ y $m_j^W = \max_k |W_{jk}|$. Con los factores de la definición 5.4,*

$$\max_i |\hat{X}_{ij}| = (m_j^X)^{1-\alpha}(m_j^W)^{1-\alpha}, \qquad \max_k |\hat{W}_{jk}| = (m_j^X)^{\alpha}(m_j^W)^{\alpha}.$$

En particular, con $\alpha = \frac{1}{2}$ ambos máximos valen $\sqrt{m_j^X m_j^W}$.

Demostración. $\hat{X}_{ij} = X_{ij}/s_j$, luego $\max_i |\hat{X}_{ij}| = m_j^X/s_j = m_j^X (m_j^W)^{1-\alpha}/(m_j^X)^{\alpha} = (m_j^X)^{1-\alpha}(m_j^W)^{1-\alpha}$. Del mismo modo, $\max_k |\hat{W}_{jk}| = s_j m_j^W = (m_j^X)^{\alpha}(m_j^W)^{\alpha}$. Con $\alpha = \frac{1}{2}$, ambas expresiones coinciden. ∎

El factor $\mathrm{diag}(s)^{-1}$ puede incorporarse de antemano a la capa anterior (por ejemplo, a los parámetros de la normalización), de modo que la transformación no cuesta nada en inferencia y permite cuantizar pesos y activaciones a int8. Los autores proponen $\alpha = 0{,}5$ como punto de equilibrio para OPT y BLOOM y otros valores para otros modelos [20, §4].

5.3 GPTQ: cuantización con información de segundo orden

Redondear cada peso a su nivel más próximo (RTN, *round-to-nearest*) ignora que los errores de pesos distintos pueden compensarse en la salida de la capa. GPTQ minimiza directamente el error de salida.

Definición 5.6 (Problema de cuantización por capas). Sea una capa lineal con pesos $W \in \mathbb{R}^{d_{\mathrm{row}} \times d_{\mathrm{col}}}$ y sea $X \in \mathbb{R}^{d_{\mathrm{col}} \times m}$ una matriz con m entradas de calibración. Se busca la matriz cuantizada $\hat{W}$ que minimiza $\|WX - \hat{W}X\|_F^2$ [6, §3, ec. 1].

El objetivo se separa por filas: para una fila w y su cuantizada $\hat{w} = w + \delta$, el error es $\|\delta X\|^2 = \delta X X^{\top}\delta^{\top} = \frac{1}{2}\,\delta H \delta^{\top}$, con $H = 2XX^{\top}$. Al ser la capa lineal, esta forma cuadrática es exacta y no una aproximación de Taylor.

Teorema 5.7 (Actualización óptima tras cuantizar un peso). *Sea H simétrica definida positiva y sea $E(\delta) = \frac{1}{2}\delta H \delta^\top$. Si se fija la componente q de la perturbación al error de redondeo, $\delta_q = \mathrm{quant}(w_q) - w_q$, la perturbación de los demás pesos que minimiza E es*

$$\delta = -\frac{w_q - \mathrm{quant}(w_q)}{[H^{-1}]_{qq}} (H^{-1})_{q,:}, \qquad \text{con error mínimo} \qquad E = \frac{\big(w_q - \mathrm{quant}(w_q)\big)^2}{2\,[H^{-1}]_{qq}}.$$

Demostración. Sea $\varepsilon = \mathrm{quant}(w_q) - w_q$. Se minimiza $\frac{1}{2}\delta H \delta^\top$ con la restricción lineal $\delta e_q = \varepsilon$, donde e_q es el q-ésimo vector de la base canónica (como columna). El lagrangiano es $\mathcal{L} = \frac{1}{2}\delta H \delta^\top + \lambda(\varepsilon - \delta e_q)$, y su gradiente respecto a δ se anula si $\delta H = \lambda e_q^\top$, es decir, $\delta = \lambda e_q^\top H^{-1} = \lambda (H^{-1})_{q,:}$ (usando la simetría de H^{-1}). Imponiendo la restricción, $\lambda [H^{-1}]_{qq} = \varepsilon$, luego $\lambda = \varepsilon / [H^{-1}]_{qq}$ y se obtiene la fórmula de δ, ya que $\varepsilon = -(w_q - \mathrm{quant}(w_q))$. Como E es convexa y la restricción lineal, este punto crítico es el mínimo. El error vale

$$E = \tfrac{1}{2}\lambda^2 (H^{-1})_{q,:} H (H^{-1})_{:,q} = \tfrac{1}{2}\lambda^2 [H^{-1}]_{qq} = \frac{\varepsilon^2}{2[H^{-1}]_{qq}}. \qquad \blacksquare$$

Este resultado es la fórmula del *Optimal Brain Surgeon* que usa el método OBQ [6, §3, ec. 2]. GPTQ lo hace aplicable a modelos de cientos de miles de millones de parámetros con tres ideas [6, §4]: cuantizar las columnas en un orden fijo, igual para todas las filas, en lugar de elegir en cada paso el peso de menor error; aplicar las actualizaciones por bloques de 128 columnas para reducir el tráfico de memoria, y obtener las filas necesarias de H^{-1} con una factorización de Cholesky, más estable numéricamente, añadiendo a H un pequeño múltiplo de la identidad. Así cuantiza modelos de unos $175 \cdot 10^9$ parámetros a 3 o 4 bits en unas cuatro horas de una GPU, con pérdida de precisión pequeña [6].

5.4 AWQ: proteger los canales importantes

Lin *et al.* observan que conservar en fp16 alrededor del 1 % de los pesos reduce mucho el error de cuantización si esos pesos se eligen por la magnitud de las *activaciones* que los multiplican, y no por la de los propios pesos [16, §3.1]. Como la precisión mixta es incómoda para el hardware, AWQ consigue el mismo efecto con un escalado.

Proposición 5.8 (Efecto del escalado de un canal). *Sea un peso w que multiplica a una entrada x, cuantizado con el cuantizador simétrico $Q(w) = \Delta\,\mathrm{round}(w/\Delta)$. Si se escala el peso por $s > 1$ y la entrada por $1/s$, el error de la salida es*

$$Q(ws)\,\frac{x}{s} - wx = \frac{\Delta'}{s}\Big(\mathrm{round}\Big(\frac{ws}{\Delta'}\Big) - \frac{ws}{\Delta'}\Big)x,$$

donde Δ' es el paso del grupo tras el escalado. Si $\Delta' \approx \Delta$ y el error de redondeo relativo al paso tiene la misma distribución en ambos casos, el error esperado de ese canal se divide aproximadamente por s [16, §3.2, ecs. 2 y 3].

Demostración. $Q(ws)\,x/s = \Delta'\,\mathrm{round}(ws/\Delta')\,x/s$ y $wx = \Delta'(ws/\Delta')\,x/s$; restando se obtiene la expresión. El factor $\mathrm{round}(u) - u$ está en $[-\frac{1}{2}, \frac{1}{2}]$; si su distribución no cambia y $\Delta' \approx \Delta$, el error queda multiplicado por $1/s$ respecto al caso sin escalar. $\blacksquare$

La hipótesis $\Delta' \approx \Delta$ es razonable si se escala un solo canal y no es el de mayor módulo del grupo; si s es demasiado grande, el paso del grupo aumenta y empeoran los demás canales.

AWQ elige $s = s_X^{\alpha}$, con s_X la magnitud media de cada canal de activación y $\alpha \in [0, 1]$ obtenido por búsqueda en un conjunto de calibración [16, §3.2, ec. 5]. El método solo cuantiza pesos y no necesita retropropagación.

5.5 QLoRA: cuantizar a 4 bits para ajustar

QLoRA ajusta un modelo congelado en 4 bits con adaptadores de bajo rango (módulo 20) e introduce dos técnicas de cuantización [4, §3].

Definición 5.9 (NormalFloat de 4 bits). Sea $Q_{\mathcal{N}}$ la función cuantil de la normal estándar. El formato NF4 toma como niveles los valores

$$q_i = \tfrac{1}{2}\Big(Q_{\mathcal{N}}\Big(\frac{i}{2^k+1}\Big) + Q_{\mathcal{N}}\Big(\frac{i+1}{2^k+1}\Big)\Big),$$

con $k = 4$, ajustados para que el 0 se represente exactamente y normalizados al intervalo $[-1, 1]$; cada bloque de 64 pesos se divide por su máximo absoluto antes de cuantizarse [4, §3, ec. 4].

Los pesos entrenados tienen una distribución aproximadamente normal; al situar los niveles en cuantiles de la normal, cada nivel recibe una fracción parecida de los pesos, cosa que no ocurre con una rejilla uniforme.

Proposición 5.10 (Doble cuantización). *Con bloques de 64 pesos y una constante fp32 por bloque, las constantes cuestan* $32/64 = 0{,}5$ *bits por peso. Si esas constantes se cuantizan a su vez en 8 bits, en bloques de 256 con una constante fp32 por bloque, el coste baja a* $8/64 + 32/(64 \cdot 256) \approx 0{,}127$ *bits por peso, un ahorro de unos* 0,373 *bits por peso [4, §3].*

Demostración. Por la proposición 4.8, una constante de 32 bits por cada 64 pesos añade $32/64$ bits. Tras la doble cuantización, cada bloque de 64 pesos lleva una constante de 8 bits ($8/64$ bits por peso) y cada grupo de 256 constantes, que cubre $64 \cdot 256$ pesos, lleva una de 32 bits ($32/(64 \cdot 256)$ bits por peso). La suma es $0{,}125 + 0{,}00195 = 0{,}127$ y la diferencia con 0,5 es 0,373. ∎

Con estas técnicas y optimizadores paginados, QLoRA ajusta un modelo de $65 \cdot 10^9$ parámetros en una sola GPU de 48 GB [4].

5.6 Destilación de conocimiento

La destilación entrena un modelo pequeño (el *estudiante*) para imitar las distribuciones de salida de uno grande (el *profesor*), que contienen más información que las etiquetas: las probabilidades relativas de las respuestas incorrectas [10, §2].

Definición 5.11 (Pérdida de destilación). Sean $v \in \mathbb{R}^K$ los logits del profesor y $z \in \mathbb{R}^K$ los del estudiante para una misma entrada, y sea $\tau > 0$. Sean $p = \operatorname{softmax}(v/\tau)$ los *objetivos blandos* y $q = \operatorname{softmax}(z/\tau)$. La pérdida de destilación es la entropía cruzada $C = -\sum_i p_i \ln q_i$, que en la práctica se combina con la entropía cruzada respecto a las etiquetas a temperatura 1 [10, §2, ec. 1].

Proposición 5.12 (Gradiente de la pérdida de destilación). $\dfrac{\partial C}{\partial z_i} = \dfrac{1}{\tau}(q_i - p_i)$.

Demostración. $\ln q_j = z_j/\tau - \ln \sum_k e^{z_k/\tau}$, de modo que $\partial \ln q_j / \partial z_i = \frac{1}{\tau}(\mathbb{1}[i=j] - q_i)$. Entonces

$$\frac{\partial C}{\partial z_i} = -\sum_j p_j \frac{1}{\tau}\big(\mathbb{1}[i=j] - q_i\big) = -\frac{1}{\tau}\Big(p_i - q_i \sum_j p_j\Big) = \frac{1}{\tau}(q_i - p_i),$$

porque $\sum_j p_j = 1$ [10, §2, ec. 2]. ∎

Para temperaturas altas y logits de media nula, el gradiente se aproxima por $(z_i - v_i)/(K\tau^2)$: destilar equivale entonces a igualar logits por mínimos cuadrados, y los gradientes de los objetivos blandos escalan como $1/\tau^2$, por lo que se multiplican por τ^2 al combinarlos con los de las etiquetas [10, §2.1]; la demostración se pide en el ejercicio 5.7.

Observación 5.13 (Destilación e inferencia). La clase 10 de CS336 describe una receta para obtener modelos más baratos: definir una arquitectura menor (por ejemplo, podando capas, cabezas o dimensiones del original), inicializarla con los pesos del original y reparar la pérdida de calidad por destilación [9]. La destilación también se usa para obtener borradores próximos al objetivo en la decodificación especulativa (tema 3). Si los modelos de las familias Claude, Gemini o Grok que se ofrecen en tamaños menores se obtienen por destilación es, en general, información no pública salvo que el fabricante lo haya declarado.

Lecturas recomendadas para este tema

- Dettmers *et al.* [3], secciones 3 («Int8 Matrix Multiplication at Scale») y 4 («Emergent Large Magnitude Features»).
- Xiao *et al.* [20], secciones 3 y 4.
- Frantar *et al.* [6], secciones 3 («Background») y 4 («The GPTQ Algorithm»).
- Lin *et al.* [16], sección 3.
- Dettmers *et al.* [4], sección 3 («QLoRA Finetuning»).
- Hinton, Vinyals y Dean [10], sección 2 y §2.1.
- MIT 6.5940 [8], clases 6, 9 y 13.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ (a) ¿Por qué un único rasgo atípico de magnitud 60 en una fila de activaciones arruina la cuantización int8 por token de esa fila? (b) ¿Por qué el suavizado de SmoothQuant no cambia el resultado del producto ni cuesta tiempo en inferencia? (c) ¿Qué información usa GPTQ que el redondeo al más próximo ignora?

Ejercicio 5.2 ⋆⋆ Se quiere calcular $x \cdot w$ con $x = (3;\ -8;\ 5)$ y $w = (0{,}6;\ 2{,}2;\ -4)$ en int8 simétrico con escala absmax por vector, $s = 127/\|\cdot\|_\infty$, como en LLM.int8(). Cuantice ambos vectores, calcule el producto en enteros, descuantice y compare con el valor exacto.

Fuente: adaptado de University of Washington, CSE 493G1/599G1 [19], sección 4 «Quantization», ejemplo de producto escalar absmax en int8.

Ejercicio 5.3 ⋆⋆ Sean

$$X = \begin{pmatrix} 1 & -16 & 0{,}5 \\ -0{,}5 & 8 & 1 \end{pmatrix}, \qquad W = \begin{pmatrix} 0{,}25 & -0{,}1 \\ 0{,}05 & 0{,}25 \\ -0{,}5 & 1 \end{pmatrix}.$$

(a) Calcule los factores de SmoothQuant con $\alpha = \frac{1}{2}$. (b) Calcule $\hat{X}$ y $\hat{W}$, y compruebe la proposición 5.5. (c) Compruebe que $\hat{X}\hat{W} = XW$. (d) Compare el cociente entre el mayor y el menor máximo por canal de las activaciones antes y después.

Ejercicio 5.4 ⋆⋆ Una fila de pesos $w = (0{,}7;\ 0{,}3)$ se cuantiza a la rejilla de paso 0,5 (múltiplos de 0,5) con entradas de calibración $X = \begin{pmatrix} 1 & 1 \\ -1 & 0 \end{pmatrix}$ (cada columna es una entrada). (a) Calcule $H = 2XX^\top$ y H^{-1}. (b) Cuantice por redondeo al más próximo y calcule el error $\|\delta X\|^2$. (c) Aplique el teorema 5.7: cuantice w_1, actualice w_2, cuantícelo y calcule el error final. Compare.

Ejercicio 5.5 ⋆⋆ Un modelo de $65 \cdot 10^9$ parámetros se guarda en (a) fp16, (b) NF4 con una constante fp32 por bloque de 64 y (c) NF4 con doble cuantización. Calcule la memoria de los pesos en cada caso y el ahorro de la doble cuantización.

Ejercicio 5.6 ⋆⋆ Un profesor da logits $v = (3, 1, 0)$ y un estudiante $z = (2, 1, 1)$. (a) Calcule los objetivos blandos p y la distribución del estudiante q con $\tau = 1$ y con $\tau = 4$. (b) Calcule el gradiente $\partial C/\partial z$ en ambos casos. (c) Multiplique el de $\tau = 4$ por τ^2 y compárelo con el de $\tau = 1$.

Ejercicio 5.7 ⋆⋆⋆ (a) Usando $e^u \approx 1 + u$ para $|u|$ pequeño, demuestre que, si τ es grande frente a los logits y estos tienen media nula ($\sum_i z_i = \sum_i v_i = 0$), entonces $\partial C/\partial z_i \approx (z_i - v_i)/(K\tau^2)$. (b) Deduzca que, en ese régimen, minimizar C equivale a minimizar $\frac{1}{2K\tau^2}\sum_i (z_i - v_i)^2$. (c) Explique por qué conviene multiplicar la pérdida blanda por τ^2. (d) Compruebe la aproximación con los datos del ejercicio 5.6 y $\tau = 10$, centrando antes los logits.

Tema 6

Servicio a escala: latencia, rendimiento y memoria

Un servidor de inferencia recibe peticiones que llegan en momentos distintos, con prompts y respuestas de longitudes muy diferentes. Este tema define las métricas con las que se mide (tiempo hasta el primer token, tiempo por token y rendimiento), analiza por qué el *batching* estático desperdicia el acelerador y cómo lo resuelve el *batching* continuo, y estudia la gestión paginada de la caché KV, que elimina la fragmentación y permite compartir prefijos. Se siguen la clase 10 de CS336 [9], el artículo de Orca de Yu *et al.* [21], las secciones 3 y 4 de Kwon *et al.* [14] y la sección 2 de Pope *et al.* [17].

6.1 Métricas de servicio

Definición 6.1 (Latencia y rendimiento). Para una petición, el *tiempo hasta el primer token* (TTFT) es el intervalo entre su llegada y la emisión del primer token de la respuesta; incluye la espera en cola y el *prefill*. El *tiempo por token de salida* (TPOT) es el intervalo medio entre tokens sucesivos, y su inverso es la velocidad que percibe el usuario. El *rendimiento* (*throughput*) del sistema es el número total de tokens generados por segundo para todas las peticiones.

Proposición 6.2 (Latencia de una respuesta). *Una respuesta de n_{out} tokens se completa en* $\text{TTFT} + (n_{\text{out}} - 1)\,\text{TPOT}$.

Demostración. El primer token llega en el instante TTFT y cada uno de los $n_{\text{out}} - 1$ restantes llega, en media, TPOT después del anterior. ■

Ejemplo 6.3. Con TTFT = 0,2 s y 50 tokens/s (TPOT = 0,02 s), una respuesta de 300 tokens llega completa en $0{,}2 + 299 \cdot 0{,}02 = 6{,}18$ s.

Las dos métricas dependen de fases distintas: el TTFT del *prefill*, limitado por el cómputo, y el TPOT de la decodificación, limitada por la memoria (tema 2). Por eso conviene procesar el *prefill* con lotes pequeños, para reducir el TTFT, y la decodificación con lotes grandes, para aumentar el rendimiento [9]. Además, replicar el modelo en M grupos de aceleradores multiplica el rendimiento por M sin cambiar la latencia, mientras que repartir un modelo entre aceleradores (módulo 17) reduce la latencia a cambio de comunicación [9, 17].

6.2 *Batching* estático y continuo

Definición 6.4 (*Batching* estático). En el *batching* estático, el servidor forma un lote de B peticiones y lo ejecuta hasta que termina la más larga; solo entonces admite peticiones nuevas.

Proposición 6.5 (Utilización del lote estático). *Si las respuestas del lote tienen longitudes $\ell_1, \ldots, \ell_B$, la fracción de ranuras de cómputo útiles durante la decodificación es*

$$U = \frac{\sum_{i=1}^{B} \ell_i}{B \max_i \ell_i}.$$

Demostración. El lote ejecuta $\max_i \ell_i$ pasos con B ranuras cada uno, en total $B \max_i \ell_i$ ranuras; de ellas, la petición i ocupa ℓ_i y las demás quedan vacías o con relleno. ■

Definición 6.6 (*Batching* continuo). En el *batching* continuo, o planificación por iteración, el servidor decide la composición del lote en cada paso de decodificación: cuando una secuencia termina, su ranura se asigna inmediatamente a una petición en espera [21]. Como las secuencias tienen longitudes distintas, Orca aplica el *batching selectivo*: las operaciones que actúan posición a posición (proyecciones y perceptrón) se ejecutan sobre la concatenación de todas las posiciones del lote, y la atención se calcula por separado para cada secuencia [21, 9].

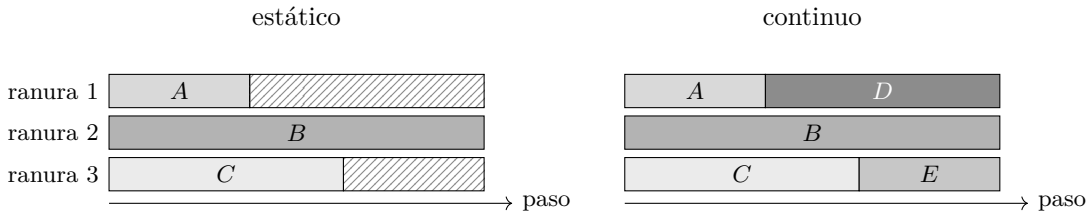


Figura 6.1: Tres ranuras durante ocho pasos de decodificación. Con *batching* estático (izquierda), las ranuras de A y C quedan vacías (rayado) hasta que termina B: se usan 16 de 24 ranuras. Con *batching* continuo (derecha), D y E entran en cuanto se liberan las ranuras y se usan las 24.

Proposición 6.7 (Utilización del *batching* continuo). *Si en todo paso hay al menos tantas peticiones en espera como ranuras liberadas, el* batching *continuo mantiene ocupadas todas las ranuras de decodificación.*

Demostración. Al final de cada paso, cada ranura liberada recibe una petición en espera, que existe por hipótesis; ninguna ranura queda vacía en el paso siguiente. ■

La proposición ignora el coste del *prefill* de las peticiones que entran, que se intercala con la decodificación de las demás; su planificación es un problema propio de los sistemas de servicio.

6.3 Memoria de la caché KV y PagedAttention

Antes de vLLM, los servidores reservaban para cada petición un bloque contiguo de caché KV del tamaño máximo posible de la respuesta. Esa reserva produce *fragmentación interna* (posiciones reservadas que nunca se usan) y *fragmentación externa* (huecos entre reservas de tamaños distintos); en los sistemas que Kwon *et al.* analizan, solo entre el 20,4 % y el 38,2 % de la memoria de la caché guarda estados de tokens [14, §1 y §3.1].

Definición 6.8 (PagedAttention). *PagedAttention* divide la caché KV de cada secuencia en *bloques* de tamaño fijo de b_{blk} tokens, que no tienen por qué ser contiguos en memoria. Cada secuencia tiene una *tabla de bloques* que traduce cada bloque lógico (posiciones $jb_{\text{blk}}, \ldots, (j+$

$1)b_{\text{blk}} - 1)$ a un bloque físico. Los bloques se asignan cuando se llenan los anteriores, y el núcleo de atención lee la caché siguiendo la tabla [14, §4.1 y §4.2].

La analogía es la memoria virtual paginada de un sistema operativo: la secuencia ve un espacio lógico contiguo, formado por páginas físicas dispersas, y la tabla de bloques hace de tabla de páginas.

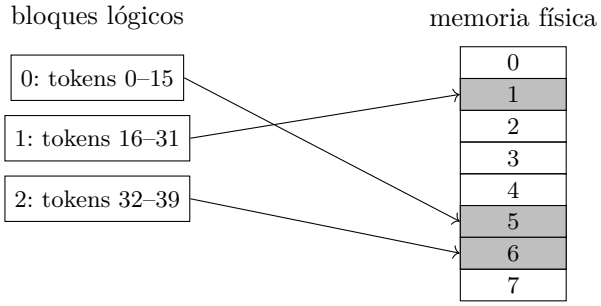


Figura 6.2: Una secuencia de 40 tokens con bloques de 16. La tabla de bloques ($0 \mapsto 5$, $1 \mapsto 1$, $2 \mapsto 6$) la sitúa en tres bloques físicos no contiguos; solo se desperdician 8 posiciones del último bloque.

Proposición 6.9 (Desperdicio acotado). *Con bloques de b_{blk} tokens, una secuencia de $\ell \geq 1$ tokens ocupa $\lceil \ell / b_{\text{blk}} \rceil$ bloques y desperdicia $b_{\text{blk}} \lceil \ell / b_{\text{blk}} \rceil - \ell \leq b_{\text{blk}} - 1$ posiciones. Como todos los bloques tienen el mismo tamaño, no hay fragmentación externa: cualquier bloque libre sirve a cualquier secuencia.*

Demostración. Por la definición de la parte entera superior, $b_{\text{blk}} \lceil \ell / b_{\text{blk}} \rceil$ es el menor múltiplo de b_{blk} mayor o igual que ℓ, y su diferencia con ℓ es menor que b_{blk}. Un bloque libre puede asignarse a cualquier entrada de cualquier tabla, porque la traducción no exige contigüidad. ■

Ejemplo 6.10. Una respuesta de 300 tokens con bloques de 16 ocupa $\lceil 300/16 \rceil = 19$ bloques, 304 posiciones, y desperdicia 4. Con una reserva contigua de 2048 posiciones desperdiciaría 1748, el 85,4 %. El tamaño de bloque por defecto de vLLM es 16: bloques más pequeños desperdician menos, pero reducen el paralelismo de la lectura de la caché [14, §7.2].

Proposición 6.11 (Compartición de prefijos). *Sean n_s secuencias que comparten un prompt de P tokens. Con compartición de bloques y* copia en escritura*, el prompt ocupa $\lfloor P / b_{\text{blk}} \rfloor$ bloques completos compartidos más, si P no es múltiplo de b_{blk}, una copia del bloque parcial por secuencia, en lugar de $n_s \lceil P / b_{\text{blk}} \rceil$ bloques.*

Demostración. Los bloques completos del prompt no se modifican nunca y pueden apuntarse desde las n_s tablas, con un contador de referencias. El último bloque, si está incompleto, recibirá tokens distintos en cada secuencia; al primer intento de escritura se copia (copia en escritura), de modo que acaba habiendo una copia por secuencia [14, §4.3 y §4.4]. ■

Este mecanismo sirve para el muestreo de varias respuestas por prompt, para la búsqueda en haz y para los prompts de sistema comunes. Las API públicas de Anthropic y de Google ofrecen una caché de prefijos del prompt (*prompt caching* y *context caching*) como función documentada; su implementación interna no es pública.

Lecturas recomendadas para este tema

- CS336 [9], clase 10 «Inference»: *throughput and latency*, *continuous batching* y *paged attention*.
- Yu *et al.* [21], planificación por iteración y *batching* selectivo.
- Kwon *et al.* [14], secciones 3 («Memory Challenges in LLM Serving») y 4 («Method»), y §7.2.
- Pope *et al.* [17], sección 2 «Inference Cost Tradeoffs».

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Explique con la analogía de la memoria virtual de un sistema operativo qué problemas resuelve PagedAttention y qué papel tiene la tabla de bloques. ¿Qué equivale a compartir una página de solo lectura entre procesos?

Ejercicio 6.2 ⋆⋆ Una petición con 1000 tokens de caché se guarda en bloques de 16. ¿Cuántos bloques ocupa y cuántas posiciones desperdicia? Compárelo con una reserva contigua de 2048 posiciones.

Ejercicio 6.3 ⋆⋆ (a) Un modelo de $1{,}3 \cdot 10^{10}$ parámetros en fp16 se sirve con lote 1 en un acelerador de 300 TFLOP/s, aprovechado al 50 % en el *prefill*, y 1 TB/s. Para un prompt de 2000 tokens y una respuesta de 500, estime el TTFT (sin cola), el TPOT y la latencia total. (b) Un lote estático de 4 peticiones tiene respuestas de 100, 400, 250 y 50 tokens. Calcule su utilización.

Ejercicio 6.4 ⋆⋆ Continúe el ejercicio 2.5. Con GQA de 8 cabezas de clave y valor, los parámetros bajan a $1{,}1338 \cdot 10^{10}$ (las proyecciones de clave y valor se reducen). Con los mismos supuestos, calcule la caché por secuencia y, para $B = 64$ y $B = 256$, la memoria total, la latencia y el rendimiento. ¿Caben en 80 GB? Compare con la atención multicabeza.

Fuente: adaptado de CS336 [9], clase 10 «Inference», apartado *reduce KV cache size* (GQA en Llama 2 13B).

Ejercicio 6.5 ⋆⋆ Cuatro muestras de una misma petición comparten un prompt de 1000 tokens, con bloques de 16 y la caché de Llama 2 7B (512 KiB por token). Calcule los bloques del prompt sin y con compartición y la memoria ahorrada.

Ejercicio 6.6 ⋆⋆⋆ (a) Suponga que la longitud ℓ de las secuencias es tal que $\ell \bmod b_{\text{blk}}$ es uniforme en $\{0, \dots, b_{\text{blk}} - 1\}$. Demuestre que el desperdicio esperado por secuencia con PagedAttention es $(b_{\text{blk}} - 1)/2$. (b) Si las longitudes son uniformes en $\{1, \dots, L_{\max}\}$ y se reservan $L_{\max}$ posiciones contiguas, demuestre que el desperdicio esperado es $(L_{\max} - 1)/2$. (c) Calcule ambos para $b_{\text{blk}} = 16$ y $L_{\max} = 2048$.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa. Se usan $e = 2{,}718$, $e^{-1} = 0{,}3679$, $e^{2} = 7{,}389$ y $e^{-2} = 0{,}1353$; 1 KiB $= 1024$ bytes, 1 MiB $= 1024^2$ bytes y 1 GiB $= 1024^3$ bytes, mientras que GB y TB son potencias de 10.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. *top-k* con $k = 50$; núcleo con $p = 0{,}9$. Caso (a): $p_{(1)} = 0{,}95$. Caso (b): 200 tokens de probabilidad 0,005.

Planteamiento. *top-k* fija el número de tokens; el núcleo fija la masa conservada (proposición 1.15) y, por la proposición 1.16, la masa eliminada es la distorsión.

Desarrollo.

1. (a) Núcleo: la acumulada del primer token es $0{,}95 \geq 0{,}9$; se conserva **1** token y se elimina una masa 0,05.
2. (a) *Top-k*: se conservan 50 tokens; los 49 últimos se reparten como mucho la masa 0,05 y son cola de baja calidad.
3. (b) Núcleo: hace falta k^* con $0{,}005\,k^* \geq 0{,}9$, es decir, $k^* = 0{,}9/0{,}005 = 180$ tokens; se elimina $1 - 0{,}9 = 0{,}1$.
4. (b) *Top-k*: los 50 tokens conservados suman $50 \cdot 0{,}005 = 0{,}25$; se elimina una masa 0,75 de opciones razonables.

Resultado. **(a) núcleo: 1 token; *top-k*: 50. (b) núcleo: 180 tokens, elimina** 0,1**; *top-k*: 50 tokens, elimina** 0,75**.** El núcleo adapta el tamaño del conjunto a la confianza del modelo.

Comprobación. en (b), TV $= 0{,}75$ para *top-k* frente a TV $= 0{,}1 \leq 1 - p$ para el núcleo, como garantiza la proposición 1.16.

Solución del ejercicio 1.2

Datos. $z = (1, 0, -1)$ para A, B, C.

Planteamiento. $p_\tau = \mathrm{softmax}(z/\tau)$ (definición 1.9); *top-k* y núcleo según la definición 1.14.

Desarrollo.

1. $\tau = 1$: $e^{1} = 2{,}718$, $e^{0} = 1$, $e^{-1} = 0{,}3679$; suma 4,086. $p_1 = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$.
2. $\tau = 0{,}5$: $z/\tau = (2, 0, -2)$; $7{,}389 + 1 + 0{,}1353 = 8{,}524$. $p_{0{,}5} = (0{,}8668;\ 0{,}1173;\ 0{,}0159)$.

3. *Top-2*: masa $0{,}6652 + 0{,}2447 = 0{,}9100$; distribución $(0{,}6652/0{,}9100;\ 0{,}2447/0{,}9100) = (0{,}7311;\ 0{,}2689)$.
4. Núcleo 0,9: acumuladas $0{,}6652 < 0{,}9$ y $0{,}9100 \geq 0{,}9$; se conservan A y B.
5. La distribución *top-2* es $e^{z_1}/(e^{z_1}+e^{z_2}) = 1/(1+e^{-(z_1-z_2)})$, que solo depende de $z_1 - z_2 = 1$: $1/(1+0{,}3679) = 0{,}7311$.

Resultado. $p_1 = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$**;** $p_{0{,}5} = (0{,}8668;\ 0{,}1173;\ 0{,}0159)$**;** ***top-2*:** $(0{,}7311;\ 0{,}2689)$**; el núcleo** 0,9 **conserva** A **y** B**.**

Comprobación. $0{,}6652 + 0{,}2447 + 0{,}0900 = 0{,}9999 \approx 1$ y $0{,}8668 + 0{,}1173 + 0{,}0159 = 1$.

Solución del ejercicio 1.3

Datos. $u = (1{,}2;\ 0{,}9;\ 0{,}1;\ -0{,}5)$; $\tau \in \{1;\ 0{,}2;\ 2\}$.

Planteamiento. $p_\tau(i) = e^{u_i/\tau} / \sum_j e^{u_j/\tau}$ y la proposición 1.11.

Desarrollo.

1. $\tau = 1$: $e^u = (3{,}320;\ 2{,}460;\ 1{,}105;\ 0{,}6065)$, suma 7,491; $p_1 = (0{,}4432;\ 0{,}3283;\ 0{,}1475;\ 0{,}0810)$.
2. $\tau = 0{,}2$: $u/\tau = (6;\ 4{,}5;\ 0{,}5;\ -2{,}5)$; $e^{u/\tau} = (403{,}4;\ 90{,}02;\ 1{,}649;\ 0{,}0821)$, suma 495,2; $p_{0{,}2} = (0{,}8147;\ 0{,}1818;\ 0{,}0033;\ 0{,}0002)$.
3. $\tau = 2$: $u/\tau = (0{,}6;\ 0{,}45;\ 0{,}05;\ -0{,}25)$; $e^{u/\tau} = (1{,}822;\ 1{,}568;\ 1{,}051;\ 0{,}7788)$, suma 5,221; $p_2 = (0{,}3490;\ 0{,}3004;\ 0{,}2014;\ 0{,}1492)$.
4. Sumas: $0{,}4432 + 0{,}3283 + 0{,}1475 + 0{,}0810 = 1{,}0000$; $0{,}8147 + 0{,}1818 + 0{,}0033 + 0{,}0002 = 1{,}0000$; $0{,}3490 + 0{,}3004 + 0{,}2014 + 0{,}1492 = 1{,}0000$.
5. (b) El máximo 1,2 es único: por la proposición 1.11, $p_\tau \to (1, 0, 0, 0)$ si $\tau \to 0$ (búsqueda voraz) y $p_\tau \to (\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$ si $\tau \to \infty$.

Resultado. $p_1 = (0{,}4432;\ 0{,}3283;\ 0{,}1475;\ 0{,}0810)$**,** $p_{0{,}2} = (0{,}8147;\ 0{,}1818;\ 0{,}0033;\ 0{,}0002)$**,** $p_2 = (0{,}3490;\ 0{,}3004;\ 0{,}2014;\ 0{,}1492)$**; voraz si** $\tau \to 0$**, uniforme si** $\tau \to \infty$**.**

Comprobación. la tendencia es la esperada: la probabilidad del primer token crece al bajar τ ($0{,}3490 < 0{,}4432 < 0{,}8147$), y p_1 coincide con los valores 0,44, 0,33, 0,15 y 0,08 de la figura 7.17 de [13].

Solución del ejercicio 1.4

Datos. $v = (1;\ 0{,}5;\ 0;\ -1)$; $\tau \in \{1;\ 0{,}5\}$; $p = 0{,}8$.

Planteamiento. softmax con temperatura y, después, núcleo: ordenar, acumular y cortar en cuanto la suma llega a 0,8.

Desarrollo.

1. $\tau = 1$: $e^v = (2{,}718;\ 1{,}649;\ 1;\ 0{,}3679)$, suma 5,735; $p_1 = (0{,}4740;\ 0{,}2875;\ 0{,}1744;\ 0{,}0641)$.
2. Acumuladas: 0,4740; 0,7615; 0,9359. Se conservan tres tokens; renormalizando por 0,9359: $(0{,}5065;\ 0{,}3072;\ 0{,}1863;\ 0)$.
3. $\tau = 0{,}5$: $v/\tau = (2;\ 1;\ 0;\ -2)$, $e^{v/\tau} = (7{,}389;\ 2{,}718;\ 1;\ 0{,}1353)$, suma 11,24; $p_{0{,}5} = (0{,}6572;\ 0{,}2418;\ 0{,}0889;\ 0{,}0120)$.

4. Acumuladas: 0,6572; 0,8990. Bastan dos tokens; renormalizando por 0,8990: (0,7311; 0,2689; 0; 0).
5. (c) Si $\tau \to 0$, la distribución tiende a $(1, 0, 0, 0)$ (proposición 1.11) y el núcleo conserva un solo token: búsqueda voraz.

Resultado. **(a)** $p_1 = (0{,}4740;\ 0{,}2875;\ 0{,}1744;\ 0{,}0641)$**,** $p_{0{,}5} = (0{,}6572;\ 0{,}2418;\ 0{,}0889;\ 0{,}0120)$**.** **(b)** $\tau = 1$**: 3 tokens,** $(0{,}5065;\ 0{,}3072;\ 0{,}1863;\ 0)$**;** $\tau = 0{,}5$**: 2 tokens,** $(0{,}7311;\ 0{,}2689;\ 0;\ 0)$**.** **(c) Voraz.**

Comprobación. con $\tau = 0{,}5$ la diferencia entre los dos primeros logits escalados es $2 - 1 = 1$ y $1/(1 + e^{-1}) = 0{,}7311$.

Solución del ejercicio 1.5

Datos. $K = 50\,000$, $T = 20$, $m = 4$; candidatas $(L = 3,\ P = 0{,}12)$ y $(L = 5,\ P = 0{,}08)$; $a = 0{,}75$.

Planteamiento. la voraz evalúa K extensiones por paso y el haz mK (proposición 1.6); la exhaustiva, K^T (proposición 1.2). Puntuación de la observación 1.7.

Desarrollo.

1. Voraz: $KT = 50\,000 \cdot 20 = 10^6$.
2. Haz: $mKT = 4 \cdot 10^6$.
3. Exhaustiva: $\log_{10} K^T = 20 \cdot 4{,}699 = 93{,}98$, luego $K^T \approx 9{,}537 \cdot 10^{93}$.
4. Candidata 1: $\ln 0{,}12 = -2{,}120$; $3^{0{,}75} = e^{0{,}75 \cdot 1{,}099} = e^{0{,}8240} = 2{,}280$; puntuación $-2{,}120/2{,}280 = -0{,}9301$.
5. Candidata 2: $\ln 0{,}08 = -2{,}526$; $5^{0{,}75} = e^{0{,}75 \cdot 1{,}609} = e^{1{,}207} = 3{,}344$; puntuación $-2{,}526/3{,}344 = -0{,}7554$.
6. Con normalización gana la 2 ($-0{,}7554 > -0{,}9301$); sin ella gana la 1 ($-2{,}120 > -2{,}526$).
7. (c) Sí: con $m \geq K^{T-1}$ el haz no poda y es la búsqueda exhaustiva; con $m = 1$ es la voraz (proposición 1.6).

Resultado. **(a)** 10^6**,** $4 \cdot 10^6$ **y** $\approx 9{,}537 \cdot 10^{93}$ **evaluaciones. (b) Con** $a = 0{,}75$ **gana la candidata larga; sin normalizar, la corta. (c) Sí, con un haz que no poda.**

Comprobación. sin normalización cada token añade un logaritmo negativo y las secuencias largas siempre salen perjudicadas, como indica la observación 1.7.

Solución del ejercicio 1.6

Datos. $P = (0{,}6;\ 0{,}3;\ 0{,}1)$; $Q \propto P^a$.

Planteamiento. por la proposición 1.10, Q es la distribución con temperatura $\tau = 1/a$.

Desarrollo.

1. (a) Duplicar la temperatura: $\tau = 2$, $a = \frac{1}{2}$. Reducirla a la mitad: $\tau = \frac{1}{2}$, $a = 2$.
2. $a = 2$: $P^2 = (0{,}36;\ 0{,}09;\ 0{,}01)$, suma 0,46; $Q = (0{,}7826;\ 0{,}1957;\ 0{,}0217)$.
3. $a = \frac{1}{2}$: $\sqrt{P} = (0{,}7746;\ 0{,}5477;\ 0{,}3162)$, suma 1,639; $Q = (0{,}4727;\ 0{,}3343;\ 0{,}1930)$.
4. (c) $\tau \to 0$ equivale a $a \to \infty$: domina el mayor $P(i)$ y $Q \to (1, 0, 0)$. $\tau \to \infty$ equivale a $a \to 0$: $P(i)^0 = 1$ y $Q \to (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.

Resultado. **(a)** $a = \frac{1}{2}$ **duplica** τ **y** $a = 2$ **la reduce a la mitad. (b)** $a = 2$**:** (0,7826; 0,1957; 0,0217)**;** $a = \frac{1}{2}$**:** (0,4727; 0,3343; 0,1930)**. (c) Voraz y uniforme.**

Comprobación. las dos distribuciones suman 1; con $a = 2$ se afila ($0{,}6 \to 0{,}7826$) y con $a = \frac{1}{2}$ se aplana ($0{,}6 \to 0{,}4727$).

Solución del ejercicio 1.7

Datos. $p_\tau(i) = e^{z_i/\tau}/Z(\tau)$ con $Z(\tau) = \sum_j e^{z_j/\tau}$; en (c), $z = (1, 0, -1)$ y $\tau = 1$.

Planteamiento. se trabaja con $\beta = 1/\tau$ y se aplica la regla de la cadena, $\frac{\mathrm{d}H}{\mathrm{d}\tau} = \frac{\mathrm{d}H}{\mathrm{d}\beta}\,\frac{\mathrm{d}\beta}{\mathrm{d}\tau}$.

Desarrollo.

1. Con $\beta = 1/\tau$, $\ln p_\tau(i) = \beta z_i - \ln Z$ y $H = -\sum_i p_\tau(i)(\beta z_i - \ln Z) = \ln Z - \beta\,\mathbb{E}[z]$, donde $\mathbb{E}[z] = \sum_i p_\tau(i) z_i$.
2. $\dfrac{\mathrm{d}\ln Z}{\mathrm{d}\beta} = \dfrac{\sum_j z_j e^{\beta z_j}}{Z} = \mathbb{E}[z]$.
3. $\dfrac{\mathrm{d}\mathbb{E}[z]}{\mathrm{d}\beta} = \sum_i z_i\, p_\tau(i)\big(z_i - \mathbb{E}[z]\big) = \mathbb{E}[z^2] - \mathbb{E}[z]^2 = \mathrm{Var}_{p_\tau}(z)$, usando $\frac{\mathrm{d}p_\tau(i)}{\mathrm{d}\beta} = p_\tau(i)(z_i - \mathbb{E}[z])$.
4. $\dfrac{\mathrm{d}H}{\mathrm{d}\beta} = \mathbb{E}[z] - \mathbb{E}[z] - \beta\,\mathrm{Var}(z) = -\beta\,\mathrm{Var}(z)$.
5. $\dfrac{\mathrm{d}\beta}{\mathrm{d}\tau} = -\dfrac{1}{\tau^2}$, luego $\dfrac{\mathrm{d}H}{\mathrm{d}\tau} = \beta\,\mathrm{Var}(z)\dfrac{1}{\tau^2} = \dfrac{\mathrm{Var}(z)}{\tau^3} \geq 0$.
6. (b) Como $p_\tau(i) > 0$ para todo i, $\mathrm{Var}_{p_\tau}(z) = 0$ si y solo si todos los z_i son iguales; entonces p_τ es uniforme para todo τ y $H = \ln K$ es constante.
7. (c) $p_1 = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$; $\mathbb{E}[z] = 0{,}6652 - 0{,}0900 = 0{,}5752$; $\mathbb{E}[z^2] = 0{,}6652 + 0{,}0900 = 0{,}7553$; $\mathrm{Var}(z) = 0{,}7553 - 0{,}5752^2 = 0{,}7553 - 0{,}3309 = 0{,}4244$.

Resultado. $\mathrm{d}H/\mathrm{d}\tau = \mathrm{Var}_{p_\tau}(z)/\tau^3 \geq 0$**; es nula solo si todos los logits son iguales; en el caso (c) vale** 0,4244**.** ■

Comprobación. una diferencia centrada, $(H(1{,}001) - H(0{,}999))/0{,}002$, da 0,4244; además, los valores del ejemplo 1.12 cumplen $H(0{,}5) = 0{,}455 < H(1) = 0{,}948 < H(2) = 1{,}245$ nats.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. el mismo Transformer; *prefill* con n tokens a la vez y decodificación con uno por paso.

Planteamiento. comparar la intensidad respecto a los pesos con la intensidad crítica I^* (proposición 2.9).

Desarrollo.

1. *Prefill*: cada peso de 2 bytes se lee una vez y se usa con los n tokens, a 2 FLOP por token: $2n/2 = n$ FLOP/byte.
2. Decodificación: cada peso se usa con un solo token: $2/2 = 1$ FLOP/byte.
3. Con $I^* = 150$ (ejemplo 2.11), un prompt de $n = 1000$ tokens supera la intensidad crítica y queda limitado por el cómputo; la decodificación, con $1 \ll 150$, por la memoria.

Resultado. **el *prefill* reutiliza cada peso n veces y queda limitado por el cómputo; la decodificación lo usa una vez y queda limitada por el ancho de banda.**

Comprobación. un prompt de un solo token tendría la misma intensidad que un paso de decodificación: las dos fases solo se distinguen por cuántos tokens comparten cada lectura de pesos.

Solución del ejercicio 2.2

Planteamiento. contrastar con el teorema 2.4, la proposición 2.12 y la proposición 2.6.

Desarrollo.

(a) **Falsa.** Por el teorema 2.4, la caché guarda exactamente lo que se recalcularía; solo cambia el coste (salvo diferencias de redondeo en coma flotante).

(b) **Falsa.** Por la proposición 2.12, $R(B) = B/(t_0 + Bt_1)$ crece menos que linealmente; en la cuadro 2.1, pasar de $B = 32$ a $B = 64$ lleva el rendimiento de 1346 a 1578 tokens/s, un factor 1,17.

(c) **Verdadera.** La caché es proporcional a n_{kv}, que se divide por $32/8 = 4$; en la misma memoria caben cuatro veces más secuencias de la misma longitud.

Resultado. **(a) falsa; (b) falsa; (c) verdadera.**

Solución del ejercicio 2.3

Datos. $L = 24$, $n_{\text{kv}} = 16$, $d_{\text{h}} = 64$, $s = 2$; $S = 2048$; $B = 16$.

Planteamiento. proposición 2.6: $c_{\text{kv}} = 2Ln_{\text{kv}}d_{\text{h}}s$; se multiplica por tokens y por secuencias.

Desarrollo.

1. $c_{\text{kv}} = 2 \cdot 24 \cdot 16 \cdot 64 \cdot 2 = 98\,304$ bytes; $98\,304/1024 = 96$ KiB.
2. Por secuencia: $96 \cdot 2048 = 196\,608$ KiB $= 196\,608/1024 = 192$ MiB.
3. Lote: $192 \cdot 16 = 3072$ MiB $= 3$ GiB.

Resultado. **96 KiB por token, 192 MiB por secuencia y 3 GiB para el lote.**

Comprobación. $98\,304 \cdot 2048 \cdot 16 = 3\,221\,225\,472$ bytes $= 3 \cdot 1024^3$.

Solución del ejercicio 2.4

Datos. $N = 1{,}3 \cdot 10^{10}$; $B_{\text{mem}} = 10^{12}$ bytes/s; $s \in \{2;\ 1;\ 0{,}5\}$.

Planteamiento. teorema 2.10 con $B = 1$: tokens/s $\leq B_{\text{mem}}/(sN)$.

Desarrollo.

1. fp16: $sN = 2{,}6 \cdot 10^{10}$ bytes; cota $10^{12}/2{,}6 \cdot 10^{10} = 38{,}46$ tokens/s.
2. int8: $sN = 1{,}3 \cdot 10^{10}$ bytes; cota 76,92 tokens/s.
3. int4: $sN = 6{,}5 \cdot 10^{9}$ bytes; cota 153,8 tokens/s.

Resultado. 38,46**,** 76,92 **y** 153,8 **tokens/s.**

Comprobación. $153{,}8/38{,}46 = 4{,}0 = 2/0{,}5$, el cociente de bytes por peso.

Solución del ejercicio 2.5

Datos. $N = 1{,}3015 \cdot 10^{10}$; bf16 ($s = 2$); $L = 40$, $n_{\text{kv}} = 40$, $d_{\text{h}} = 128$; $S = 1024$; $B_{\text{mem}} = 3{,}35 \cdot 10^{12}$ bytes/s; memoria de 80 GB.

Planteamiento. memoria $= sN + B\,S\,c_{\text{kv}}$; latencia por paso = memoria/B_{mem} (teorema 2.10); rendimiento = B/latencia.

Desarrollo.

1. $c_{\text{kv}} = 2 \cdot 40 \cdot 40 \cdot 128 \cdot 2 = 819\,200$ bytes; por secuencia, $1024 \cdot 819\,200 = 8{,}389 \cdot 10^8$ bytes $= 0{,}8389$ GB.
2. Pesos: $2 \cdot 1{,}3015 \cdot 10^{10} = 26{,}03$ GB.
3. $B = 1$: memoria $26{,}03 + 0{,}84 = 26{,}87$ GB; latencia $26{,}87 \cdot 10^9 / 3{,}35 \cdot 10^{12} = 8{,}02$ ms; rendimiento $1/0{,}00802 = 124{,}7$ tokens/s.
4. $B = 64$: caché $64 \cdot 0{,}8389 = 53{,}69$ GB; memoria 79,72 GB; latencia 23,80 ms; rendimiento $64/0{,}02380 = 2689$ tokens/s.
5. $B = 256$: caché 214,7 GB; memoria 240,8 GB; latencia 71,87 ms; rendimiento $256/0{,}07187 = 3562$ tokens/s.
6. (c) $B = 1$ y $B = 64$ caben en 80 GB (este último por muy poco); $B = 256$ no.

Resultado. **caché de** 0,839 **GB por secuencia;** (B, **memoria**, **latencia**, **rendimiento**) = (1; 26,87 **GB**; 8,02 **ms**; 124,7)**,** (64; 79,72; 23,80; 2689)**,** (256; 240,8; 71,87; 3562)**.** Aumentar el lote empeora la latencia y mejora el rendimiento con rendimientos decrecientes; con $B = 256$ la caché no cabe en la GPU.

Comprobación. el límite de la proposición 2.12 es $B_{\text{mem}}/(S\,c_{\text{kv}}) = 3{,}35 \cdot 10^{12}/8{,}389 \cdot 10^8 = 3993$ tokens/s, por encima de los tres valores y cada vez más próximo.

Solución del ejercicio 2.6

Datos. lote B; pesos en fp16 (2 bytes); $I^* = 150$ FLOP/byte.

Planteamiento. contar, para un peso, las operaciones y los bytes en un paso.

Desarrollo.

1. En un paso, cada peso w se lee una vez (2 bytes) y participa en una multiplicación y una suma por cada una de las B secuencias: $2B$ FLOP.
2. Intensidad respecto a los pesos: $2B/2 = B$ FLOP/byte. ■
3. Deja de estar limitado por los pesos si $B \geq I^* = 150$.
4. La caché KV es distinta para cada secuencia: cada número de la caché se lee en cada paso y se usa con una sola consulta, con intensidad menor que 1 (proposición 2.9(b)).
5. Su tamaño crece con B: con Llama 2 7B y 2000 tokens, para $B = 64$ ya son 67,1 GB frente a 14 GB de pesos (cuadro 2.1), y puede no caber en la memoria.

Resultado. **la intensidad es B FLOP/byte; harían falta unas 150 secuencias por lote, pero la caché, con intensidad menor que 1 y tamaño proporcional a B, domina antes el tráfico y la memoria.**

Comprobación. con $B = 1$ se recupera la intensidad de 1 FLOP/byte del ejemplo 2.11.

Solución del ejercicio 2.7

Datos. una capa de atención de dimensión D; prompt vacío; T tokens generados.

Planteamiento. el producto de a consultas por b claves cuesta $2abD$ FLOP, y el de los pesos de atención por los valores, otros $2abD$ (proposición 2.9).

Desarrollo.

1. Sin caché, el paso t recalcula la atención completa sobre t posiciones: $a = b = t$, coste $4Dt^2$.
2. Total: $\sum_{t=1}^{T} 4Dt^2 = 4D\frac{T(T+1)(2T+1)}{6} = \frac{2}{3}DT(T+1)(2T+1) = \Theta(T^3)$.
3. Con caché, el paso t calcula una consulta frente a t claves: $a = 1$, $b = t$, coste $4Dt$.
4. Total: $\sum_{t=1}^{T} 4Dt = 4D\frac{T(T+1)}{2} = 2DT(T+1) = \Theta(T^2)$. ■
5. Cociente: $\frac{\frac{2}{3}DT(T+1)(2T+1)}{2DT(T+1)} = \frac{2T+1}{3}$; con $T = 1000$, $2001/3 = 667$.

Resultado. $\Theta(T^3)$ **sin caché frente a** $\Theta(T^2)$ **con caché; para** $T = 1000$ **la caché reduce el coste de la atención unas 667 veces.**

Comprobación. para $T = 1$ ambos costes valen $4D$ y el cociente es $3/3 = 1$, como debe ser: con un solo token no hay nada que reutilizar.

A.3 Tema 3

Solución del ejercicio 3.1

Planteamiento. contrastar con los teoremas 3.4 y 3.7 y la definición 3.1.

Desarrollo.

(a) **Falsa.** Por el teorema 3.7, la salida tiene exactamente la distribución del objetivo; solo cambia el tiempo.

(b) **Verdadera.** Si $q = p$, $\min(1, p/q) = 1$ para todo token y $\beta = 1$ (lema 3.3).

(c) **Verdadera.** Si se rechaza la primera propuesta se emite un token del residuo; si no, al menos esa propuesta. Si se aceptan todas se emiten $\gamma + 1$.

Resultado. **(a) falsa; (b) verdadera; (c) verdadera.**

Solución del ejercicio 3.2

Datos. $p = (0{,}6;\ 0{,}4)$, $q = (0{,}9;\ 0{,}1)$.

Planteamiento. aceptación $\min(1, p/q)$; $\beta = \sum \min(p, q)$; residuo $\max(0, p - q)$ normalizado; $P(\text{emitir } x) = \min(p, q) + (1 - \beta)r(x)$.

Desarrollo.

1. Aceptar A: $\min(1;\ 0{,}6/0{,}9) = 2/3 \approx 0{,}6667$. Aceptar B: $\min(1;\ 0{,}4/0{,}1) = \min(1; 4) = 1$.
2. $\beta = \min(0{,}6; 0{,}9) + \min(0{,}4; 0{,}1) = 0{,}6 + 0{,}1 = 0{,}7$.
3. Residuo: $\max(0; -0{,}3) = 0$ y $\max(0; 0{,}3) = 0{,}3$; normalizado, $(0;\ 1)$.
4. $P(A) = 0{,}9 \cdot \frac{2}{3} = 0{,}6$; $P(B) = 0{,}1 \cdot 1 + 0{,}3 \cdot 1 = 0{,}4$.

Resultado. **aceptación** $2/3$ **para** A **y** 1 **para** B**;** $\beta = 0{,}7$**; residuo** $(0, 1)$**; salida** $(0{,}6;\ 0{,}4) = p$**.**

Comprobación. $1 - \beta = 0{,}3 = \frac{1}{2}(|0{,}6 - 0{,}9| + |0{,}4 - 0{,}1|)$, como afirma la proposición 3.5.

Solución del ejercicio 3.3

Datos. $\mathcal{V} = \{A, B\}$; $q(A) > p(A)$; en (c), $p = (0{,}3;\ 0{,}7)$ y $q = (0{,}8;\ 0{,}2)$.

Planteamiento. se aplica la regla de aceptación y rechazo token a token.

Desarrollo.

1. (a) $q(B) = 1 - q(A) < 1 - p(A) = p(B)$. Residuo: $\max(0, p(A) - q(A)) = 0$ y $\max(0, p(B) - q(B)) = p(B) - q(B) > 0$; normalizado, $(0, 1)$.
2. (b) Se propone A con probabilidad $q(A)$ y se acepta con $p(A)/q(A) < 1$: $P(\text{emitir } A) = q(A) \cdot \frac{p(A)}{q(A)} + 0 = p(A)$, porque el residuo nunca da A.
3. Se emite B si se propone y se acepta (probabilidad $q(B) \cdot 1$) o si se propone A, se rechaza y el residuo da B (probabilidad $q(A)\big(1 - \frac{p(A)}{q(A)}\big) \cdot 1 = q(A) - p(A)$).
4. $P(\text{emitir } B) = q(B) + q(A) - p(A) = 1 - p(A) = p(B)$.
5. (c) Aceptar A: $0{,}3/0{,}8 = 0{,}375$; $P(A) = 0{,}8 \cdot 0{,}375 = 0{,}3$; $P(B) = 0{,}2 + 0{,}8 - 0{,}3 = 0{,}7$.

Resultado. $P($**emitir** $A) = p(A)$ **y** $P($**emitir** $B) = p(B)$**; en (c),** $(0{,}3;\ 0{,}7)$**.**

Comprobación. $\beta = \min(0{,}3; 0{,}8) + \min(0{,}7; 0{,}2) = 0{,}5$ y $\mathrm{TV}(p, q) = \frac{1}{2}(0{,}5 + 0{,}5) = 0{,}5 = 1 - \beta$.

Solución del ejercicio 3.4

Datos. $\alpha = 0{,}5$; $\gamma \in \{2, 3, 10\}$; $c = 0{,}1$.

Planteamiento. $\mathbb{E}[N_{\mathrm{r}}] = (1 - \alpha^{\gamma+1})/(1 - \alpha)$ (teorema 3.10) y $f(\gamma) = \mathbb{E}[N_{\mathrm{r}}]/(\gamma c + 1)$ (teorema 3.11).

Desarrollo.

1. $\gamma = 3$: $0{,}5^4 = 0{,}0625$; $\mathbb{E}[N_{\mathrm{r}}] = 0{,}9375/0{,}5 = 1{,}875$.
2. $\gamma = 10$: $0{,}5^{11} = 0{,}000488$; $\mathbb{E}[N_{\mathrm{r}}] = 0{,}9995/0{,}5 = 1{,}999$.
3. $\gamma = 2$: $0{,}5^3 = 0{,}125$; $\mathbb{E}[N_{\mathrm{r}}] = 0{,}875/0{,}5 = 1{,}75$.
4. Factores: $f(2) = 1{,}75/1{,}2 = 1{,}458$; $f(3) = 1{,}875/1{,}3 = 1{,}442$; $f(10) = 1{,}999/2 = 0{,}9995$.

Resultado. $1{,}875$ **y** $1{,}999$ **tokens por ronda;** $f(2) = 1{,}458$**,** $f(3) = 1{,}442$ **y** $f(10) = 0{,}9995$**.** Con $\gamma = 10$ se ganan $0{,}124$ tokens por ronda a cambio de 7 pasadas del borrador, y el resultado es más lento que no especular. Con α bajo conviene γ pequeño; aquí el óptimo es $\gamma = 2$.

Comprobación. el límite de $\mathbb{E}[N_{\mathrm{r}}]$ es $1/(1 - \alpha) = 2$, y con $\gamma = 3$ ya se tiene el $93{,}75\,\%$ de ese máximo.

Solución del ejercicio 3.5

Datos. $\tilde{x} \sim q$; $\beta = \sum_x \min(p(x), q(x))$; p y q son distribuciones.

Planteamiento. (a) sin rechazo la salida es la propuesta; (b) escribir $1 - \beta$ como suma de partes positivas.

Desarrollo.

1. (a) Si se acepta siempre, el token emitido es $\tilde{x}$, y $P(\text{emitir } x) = q(x)$, que solo coincide con p si $p = q$.
2. (b) Por el lema 3.3, $1 - \beta = \sum_x \max(0, p(x) - q(x))$.
3. Como $\sum_x (p - q) = 1 - 1 = 0$, las partes positiva y negativa suman lo mismo: $\sum_x \max(0, p - q) = \sum_x \max(0, q - p)$.
4. Para cada x, $|p-q| = \max(0, p-q) + \max(0, q-p)$; sumando, $\sum_x |p-q| = 2 \sum_x \max(0, p-q)$.
5. Luego $1 - \beta = \frac{1}{2} \sum_x |p(x) - q(x)| = \mathrm{TV}(p, q)$. ■

Resultado. **sin rechazo la salida sigue q; la probabilidad de rechazo es $1-\beta = \mathrm{TV}(p, q)$.**

Comprobación. en el ejemplo 3.6, $\frac{1}{2}(0{,}3 + 0{,}3 + 0) = 0{,}3 = 1 - 0{,}7$.

Solución del ejercicio 3.6

Datos. $0 < \alpha < 1$, $c > 0$; $\mathbb{E}[N_r](\gamma) = \sum_{j=0}^{\gamma} \alpha^j$; $f(\gamma) = \mathbb{E}[N_r](\gamma)/(\gamma c + 1)$.

Planteamiento. acotar la suma geométrica y estudiar el cociente.

Desarrollo.

1. (a) $\mathbb{E}[N_r](\gamma + 1) - \mathbb{E}[N_r](\gamma) = \alpha^{\gamma+1} > 0$, luego es estrictamente creciente; y $\sum_{j=0}^{\gamma} \alpha^j < \sum_{j=0}^{\infty} \alpha^j = 1/(1 - \alpha)$.
2. (b) $0 < f(\gamma) < \dfrac{1}{(1-\alpha)(\gamma c + 1)} \to 0$ cuando $\gamma \to \infty$.
3. Sea $f_1 = f(1) > 0$. Existe γ_0 tal que $f(\gamma) < f_1$ para $\gamma > \gamma_0$; el máximo de f sobre $\{1, \ldots, \gamma_0\}$, que existe por ser un conjunto finito, es el máximo global.
4. (c) Como $\alpha^j \le \alpha$ para $j \ge 1$, $\mathbb{E}[N_r](\gamma) \le 1 + \gamma\alpha \le 1 + \gamma c$ si $\alpha \le c$, y entonces $f(\gamma) \le 1$. ■

Resultado. $\mathbb{E}[N_r]$ **crece estrictamente y está acotado por $1/(1-\alpha)$; $f(\gamma) \to 0$ y hay un γ óptimo finito; si $\alpha \le c$, especular nunca compensa.** Junto con el corolario 3.12, la condición $\alpha > c$ es necesaria y suficiente para que exista algún γ con $f(\gamma) > 1$.

Comprobación. con $\alpha = 0{,}5$ y $c = 0{,}1$ (ejercicio 3.4), f vale 1,364; 1,458; 1,442 para $\gamma = 1, 2, 3$ y 0,9995 para $\gamma = 10$: sube, alcanza el máximo en $\gamma = 2$ y decrece.

A.4 Tema 4

Solución del ejercicio 4.1

Planteamiento. relacionar cada pregunta con las definiciones 4.1, 4.2 y 4.7 y el teorema 4.3.

Desarrollo.

(a) Los pesos se distribuyen de forma aproximadamente simétrica alrededor de 0; el cuantizador simétrico aprovecha bien sus niveles y evita el término de punto cero en el producto (proposición 4.10). Una activación no negativa, como la salida de una ReLU, usaría solo la mitad de los niveles simétricos; el afín coloca los 2^b niveles en $[0, x_{\max}]$ y reduce el paso a la mitad.

(b) Los ceros son muy frecuentes (relleno, activaciones nulas, pesos podados); si el 0 se representa exactamente, no añaden error ni sesgo.

(c) La escala por canal se adapta al rango de cada canal: un canal de valores pequeños no hereda el paso grande impuesto por otro con valores grandes (proposición 4.9), y la descuantización sigue siendo un reescalado del producto entero (proposición 4.11).

Resultado. **simétrica para pesos centrados y afín para rangos asimétricos; el 0 exacto evita errores en valores muy frecuentes; la escala por canal reduce el error sin romper la aritmética entera.**

Solución del ejercicio 4.2

Datos. $x = (-1;\ 0;\ 0{,}62;\ 2)$; $b = 4$, enteros $0, \dots, 15$.

Planteamiento. definición 4.1 con calibración por mínimo y máximo.

Desarrollo.

1. $\Delta = (2 - (-1))/15 = 3/15 = 0{,}2$.
2. $Z = \text{round}(1/0{,}2) = \text{round}(5) = 5$.
3. $x/\Delta = (-5;\ 0;\ 3{,}1;\ 10)$; redondeo: $(-5;\ 0;\ 3;\ 10)$.
4. $Q = (0;\ 5;\ 8;\ 15)$; todos en $[0, 15]$, sin recorte.
5. $\hat{x} = 0{,}2 \cdot (-5;\ 0;\ 3;\ 10) = (-1;\ 0;\ 0{,}6;\ 2)$; errores $(0;\ 0;\ 0{,}02;\ 0)$.
6. $x = 3$: $\text{round}(3/0{,}2) + 5 = 15 + 5 = 20$, recortado a 15; $\hat{x} = 0{,}2 \cdot 10 = 2$ y el error es 1.

Resultado. $\Delta = 0{,}2$**;** $Z = 5$**;** $Q = (0; 5; 8; 15)$**;** $\hat{x} = (-1; 0; 0{,}6; 2)$**; error máximo** $0{,}02$**. Con** $x = 3$ **hay saturación y el error es** 1**.**

Comprobación. $0{,}02 \leq \Delta/2 = 0{,}1$; el 0 se reconstruye exactamente y el error de saturación, $1 > 0{,}1$, no respeta la cota, como prevé el teorema 4.3.

Solución del ejercicio 4.3

Datos. la matriz W; $b = 3$, enteros $0, \dots, 7$; $x_{\min} = -1{,}6$, $x_{\max} = 2{,}6$.

Planteamiento. definición 4.1 por tensor.

Desarrollo.

1. $\Delta = (2{,}6 + 1{,}6)/7 = 4{,}2/7 = 0{,}6$; $Z = \text{round}(1{,}6/0{,}6) = \text{round}(2{,}667) = 3$.
2. W/Δ y su redondeo:

$$\frac{W}{\Delta} = \begin{pmatrix} 2 & -0{,}6667 & 0{,}4167 \\ -1{,}667 & 4{,}333 & 0 \\ 0{,}8333 & -2{,}667 & 3{,}167 \end{pmatrix}, \qquad \text{round}\Big(\frac{W}{\Delta}\Big) = \begin{pmatrix} 2 & -1 & 0 \\ -2 & 4 & 0 \\ 1 & -3 & 3 \end{pmatrix}.$$

3. Sumando $Z = 3$ (sin recorte) y reconstruyendo con $\hat{W} = 0{,}6\ \text{round}(W/\Delta)$:

$$Q = \begin{pmatrix} 5 & 2 & 3 \\ 1 & 7 & 3 \\ 4 & 0 & 6 \end{pmatrix}, \qquad \hat{W} = \begin{pmatrix} 1{,}2 & -0{,}6 & 0 \\ -1{,}2 & 2{,}4 & 0 \\ 0{,}6 & -1{,}8 & 1{,}8 \end{pmatrix}.$$

4. Errores:
$$W - \hat{W} = \begin{pmatrix} 0 & 0{,}2 & 0{,}25 \\ 0{,}2 & 0{,}2 & 0 \\ -0{,}1 & 0{,}2 & 0{,}1 \end{pmatrix}.$$

Resultado. $\Delta = 0{,}6$**;** $Z = 3$**; error máximo** $0{,}25$**, en el elemento** $0{,}25$**, que se reconstruye como 0.**

Comprobación. $0{,}25 \leq \Delta/2 = 0{,}3$. El extremo $-1{,}6$ no se reconstruye exactamente $(-1{,}8)$ porque Z se redondeó de 2,667 a 3, pero su error, 0,2, respeta la cota, como garantiza el teorema 4.3 también con Z redondeado.

Solución del ejercicio 4.4

Datos. $v = (0{,}12;\ -0{,}31;\ 0{,}05;\ 0{,}27;\ -0{,}08;\ 0{,}19;\ -0{,}22;\ 6{,}4)$; int4 simétrico, $\Delta = \max|v|/7$.

Planteamiento. definición 4.2 con distintos grupos.

Desarrollo.

1. (a) Por tensor: $\Delta = 6{,}4/7 = 0{,}9143$. Todos los valores salvo el atípico cumplen $|v_i| < \Delta/2 = 0{,}457$ y se cuantizan a 0 (proposición 4.9); el atípico da $Q = 7$ y $\hat{v} = 6{,}4$. Error máximo: 0,31.
2. (b) Grupo 1, $(0{,}12;\ -0{,}31;\ 0{,}05;\ 0{,}27)$: $\Delta = 0{,}31/7 = 0{,}04429$; $v/\Delta = (2{,}71;\ -7;\ 1{,}13;\ 6{,}10)$, $Q = (3;\ -7;\ 1;\ 6)$, $\hat{v} = (0{,}1329;\ -0{,}31;\ 0{,}0443;\ 0{,}2657)$; errores $(0{,}0129;\ 0;\ 0{,}0057;\ 0{,}0043)$.
3. Grupo 2, $(-0{,}08;\ 0{,}19;\ -0{,}22;\ 6{,}4)$: $\Delta = 0{,}9143$; los tres primeros se anulan; errores $(0{,}08;\ 0{,}19;\ 0{,}22;\ 0)$. Error máximo global: 0,22.
4. (c) Sin el atípico, $\Delta = 0{,}31/7 = 0{,}04429$ para los 7 valores; $Q = (3;\ -7;\ 1;\ 6;\ -2;\ 4;\ -5)$; $\hat{v} = (0{,}1329;\ -0{,}31;\ 0{,}0443;\ 0{,}2657;\ -0{,}0886;\ 0{,}1771;\ -0{,}2214)$; error máximo 0,0129, y el atípico se guarda exacto en fp16.

Resultado. **errores máximos:** $0{,}31$ **por tensor,** $0{,}22$ **por grupos de 4 y** $0{,}0129$ **separando el atípico.** Los grupos solo ayudan a los que no comparten grupo con el atípico; separarlo, como hace LLM.int8(), resuelve el problema.

Comprobación. en (c), $0{,}0129 \leq \Delta/2 = 0{,}0221$; en (a), la cota $\Delta/2 = 0{,}457$ es 20,6 veces mayor que la de (c), igual que el cociente de escalas $6{,}4/0{,}31 = 20{,}6$.

Solución del ejercicio 4.5

Datos. enteros en $[-127, 127]$; acumulador hasta $2^{31} - 1 = 2\,147\,483\,647$; escalas fp16 (16 bits); grupos de 64 y 128; $N = 7 \cdot 10^9$.

Planteamiento. cada producto vale como mucho 127^2 en módulo; proposición 4.8 para los bits.

Desarrollo.

1. $127^2 = 16\,129$. La suma de n productos cumple $|\sum| \leq 16\,129\,n$, que no desborda si $16\,129\,n \leq 2\,147\,483\,647$, es decir, $n \leq 133\,144{,}25$.
2. Mayor n: 133 144.
3. Grupos de 64: $4 + 16/64 = 4{,}25$ bits por peso; memoria $7 \cdot 10^9 \cdot 4{,}25/8 = 3{,}719 \cdot 10^9$ bytes.
4. Grupos de 128: $4 + 16/128 = 4{,}125$ bits; memoria $7 \cdot 10^9 \cdot 4{,}125/8 = 3{,}609 \cdot 10^9$ bytes.

Resultado. **(a)** $n \leq 133\,144$**. (b)** 4,25 **bits y** 3,72 **GB con grupos de 64;** 4,125 **bits y** 3,61 **GB con grupos de 128.**

Comprobación. las dimensiones internas de los LLM (unos miles) quedan muy por debajo de 133 144, y 3,61 GB coincide con la cuadro 4.1.

Solución del ejercicio 4.6

Datos. $e \sim \mathcal{U}[-\Delta/2, \Delta/2]$; señal $x \sim \mathcal{U}[-A, A]$; $\Delta = 2A/2^b$.

Planteamiento. calcular los momentos con la densidad uniforme $1/\Delta$ y dividir varianzas.

Desarrollo.

1. $\mathbb{E}[e] = \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e\,\mathrm{d}e = 0$, por simetría del integrando impar.
2. $\mathbb{E}[e^2] = \frac{1}{\Delta}\left[\frac{e^3}{3}\right]_{-\Delta/2}^{\Delta/2} = \frac{1}{\Delta}\cdot\frac{2}{3}\cdot\frac{\Delta^3}{8} = \frac{\Delta^2}{12}$. ■
3. Varianza de la señal: con el mismo cálculo en $[-A, A]$, $\mathbb{E}[x^2] = \frac{(2A)^2}{12} = \frac{A^2}{3}$.
4. Relación señal-ruido: $\dfrac{A^2/3}{\Delta^2/12} = \dfrac{4A^2}{\Delta^2} = \dfrac{4A^2}{4A^2/2^{2b}} = 2^{2b}$.
5. En decibelios: $10\log_{10} 2^{2b} = 20\,b\log_{10} 2 = 6{,}0206\,b$. ■
6. $b = 4$: 24,08 dB; $b = 8$: 48,16 dB.

Resultado. $\mathbb{E}[e] = 0$**,** $\mathbb{E}[e^2] = \Delta^2/12$**; relación señal-ruido** 2^{2b}**, es decir,** $6{,}02\,b$ **dB:** 24,08 **dB con 4 bits y** 48,16 **dB con 8.**

Comprobación. la simulación del texto ($\Delta = 0{,}01$) da $8{,}345 \cdot 10^{-6}$ frente a $8{,}333 \cdot 10^{-6}$, y la diferencia entre 8 y 4 bits es $4 \cdot 6{,}02 = 24{,}08$ dB, conforme al corolario 4.6.

Solución del ejercicio 4.7

Datos. cuantización simétrica; en (b), $X = (1\ 4)$, $W = (1\ 1)^\top$, $\Delta^X = (1, 4)$, $\Delta_W = 1$.

Planteamiento. comparar los máximos que definen las escalas; en (b), evaluar el sumatorio con y sin las escalas dentro.

Desarrollo.

1. (a) Por tensor, $\Delta = \max_{k,j}|W_{kj}|/(2^{b-1}-1)$; por canal, $\Delta_j = \max_k|W_{kj}|/(2^{b-1}-1)$.
2. Para todo j, $\max_k|W_{kj}| \leq \max_{k,j'}|W_{kj'}|$, luego $\Delta_j \leq \Delta$ y $\Delta_j/2 \leq \Delta/2$. ■
3. (b) $Q_X = (1/1,\ 4/4) = (1, 1)$ y $Q_W = (1, 1)^\top$. Con las escalas dentro del sumatorio: $1\cdot1\cdot1\cdot1 + 4\cdot1\cdot1\cdot1 = 5 = XW$.
4. Producto entero: $Q_X Q_W = 1 + 1 = 2$. Multiplicado por $\Delta_W = 1$ y por una escala de X: $2\cdot1 = 2$ o $2\cdot4 = 8$; ninguno es 5.

Resultado. **(a) la cota por canal nunca supera la cota por tensor. (b) El resultado correcto, 5, exige ponderar cada término por su escala** Δ_k^X**; un reescalado único del producto entero da 2 u 8.**

Comprobación. la igualdad en (a) se da en el canal que contiene el máximo global, que es el único que no mejora.

A.5 Tema 5

Solución del ejercicio 5.1

Planteamiento. relacionar con la proposición 4.9, la proposición 5.3 y la definición 5.6.

Desarrollo.

(a) Con absmax por token, $\Delta = 60/127 = 0{,}472$; todos los valores de la fila con $|x| < 0{,}236$ se cuantizan a 0 y los demás usan muy pocos niveles (proposición 4.9). Como los atípicos aparecen en casi todas las filas y en las mismas dimensiones, el problema afecta a toda la matriz.

(b) Por la proposición 5.3, $XW = (X\,\mathrm{diag}(s)^{-1})(\mathrm{diag}(s)W)$ exactamente. $\mathrm{diag}(s)W$ se calcula una vez antes de servir el modelo, y $\mathrm{diag}(s)^{-1}$ se incorpora a los parámetros de la capa anterior, de modo que no hay operaciones nuevas en inferencia.

(c) GPTQ usa la matriz $H = 2XX^\top$, construida con datos de calibración, que describe cómo se combinan los errores de los pesos en la salida; el redondeo al más próximo trata cada peso por separado y no corrige los demás.

Resultado. **(a) el atípico infla la escala de toda la fila; (b) es una identidad que se precalcula; (c) la información de segundo orden de las entradas.**

Solución del ejercicio 5.2

Datos. $x = (3;\ -8;\ 5)$, $w = (0{,}6;\ 2{,}2;\ -4)$; enteros en $[-127, 127]$.

Planteamiento. $s_x = 127/\|x\|_\infty$, $s_w = 127/\|w\|_\infty$; $Q = \mathrm{round}(s\cdot\mathrm{valor})$; $x\cdot w \approx (Q_x\cdot Q_w)/(s_x s_w)$ (definición 4.2, proposición 4.11).

Desarrollo.

1. $s_x = 127/8 = 15{,}875$; $s_w = 127/4 = 31{,}75$.
2. $s_x x = (47{,}63;\ -127;\ 79{,}38)$, $Q_x = (48;\ -127;\ 79)$.
3. $s_w w = (19{,}05;\ 69{,}85;\ -127)$, $Q_w = (19;\ 70;\ -127)$.
4. Productos: $48\cdot 19 = 912$; $-127\cdot 70 = -8890$; $79\cdot(-127) = -10\,033$. Suma en int32: $-18\,011$.
5. Escala conjunta: $15{,}875\cdot 31{,}75 = 504{,}0$. Descuantizado: $-18\,011/504{,}0 = -35{,}73$.
6. Exacto: $1{,}8 - 17{,}6 - 20 = -35{,}8$.

Resultado. $x\cdot w \approx -35{,}73$ **en int8 frente a** $-35{,}8$ **exacto: error relativo** $0{,}07/35{,}8 \approx 0{,}19\,\%$**.**

Comprobación. los elementos de mayor módulo se cuantizan a ± 127 sin error; los demás tienen error menor que medio paso, $0{,}5/15{,}875 = 0{,}0315$ en x y $0{,}5/31{,}75 = 0{,}0157$ en w.

Solución del ejercicio 5.3

Datos. las matrices X (2×3) y W (3×2); $\alpha = \frac{1}{2}$.

Planteamiento. definición 5.4: $s_j = \sqrt{m_j^X / m_j^W}$; después $\hat{X} = X\,\mathrm{diag}(s)^{-1}$ y $\hat{W} = \mathrm{diag}(s)W$.

Desarrollo.

1. Máximos por canal: $m^X = (1;\ 16;\ 1)$ (columnas de X) y $m^W = (0{,}25;\ 0{,}25;\ 1)$ (filas de W).
2. $s = (\sqrt{1/0{,}25};\ \sqrt{16/0{,}25};\ \sqrt{1/1}) = (2;\ 8;\ 1)$.
3. Dividiendo las columnas de X por s y multiplicando las filas de W por s:

$$\hat{X} = \begin{pmatrix} 0{,}5 & -2 & 0{,}5 \\ -0{,}25 & 1 & 1 \end{pmatrix}, \qquad \hat{W} = \begin{pmatrix} 0{,}5 & -0{,}2 \\ 0{,}4 & 2 \\ -0{,}5 & 1 \end{pmatrix}.$$

4. Máximos tras el suavizado: $(0{,}5;\ 2;\ 1)$ en ambos, igual a $\sqrt{m_j^X m_j^W} = (\sqrt{0{,}25};\ \sqrt{4};\ \sqrt{1})$, como afirma la proposición 5.5.
5. XW: primera fila $(0{,}25 - 0{,}8 - 0{,}25;\ -0{,}1 - 4 + 0{,}5) = (-0{,}8;\ -3{,}6)$; segunda $(-0{,}125 + 0{,}4 - 0{,}5;\ 0{,}05 + 2 + 1) = (-0{,}225;\ 3{,}05)$.
6. $\hat{X}\hat{W}$: primera fila $(0{,}25 - 0{,}8 - 0{,}25;\ -0{,}1 - 4 + 0{,}5) = (-0{,}8;\ -3{,}6)$; segunda $(-0{,}125 + 0{,}4 - 0{,}5;\ 0{,}05 + 2 + 1) = (-0{,}225;\ 3{,}05)$.
7. Cociente entre máximos por canal de las activaciones: antes $16/1 = 16$; después $2/0{,}5 = 4$.

Resultado. $s = (2;\ 8;\ 1)$**; máximos por canal** $(0{,}5;\ 2;\ 1)$ **en** $\hat{X}$ **y en** $\hat{W}$**;** $\hat{X}\hat{W} = XW = \begin{pmatrix} -0{,}8 & -3{,}6 \\ -0{,}225 & 3{,}05 \end{pmatrix}$**; el desequilibrio entre canales de las activaciones baja de 16 a 4.**

Comprobación. el canal atípico (el segundo) se divide por $s_2 = 8$ en X y su fila de W se multiplica por 8: el producto de ambos máximos, $16 \cdot 0{,}25 = 4 = 2 \cdot 2$, se conserva.

Solución del ejercicio 5.4

Datos. $w = (0{,}7;\ 0{,}3)$; rejilla de paso 0,5; $X = \begin{pmatrix} 1 & 1 \\ -1 & 0 \end{pmatrix}$.

Planteamiento. error $E(\delta) = \|\delta X\|^2$; actualización del teorema 5.7 tras cuantizar w_1.

Desarrollo.

1. $XX^\top = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}$, $H = \begin{pmatrix} 4 & -2 \\ -2 & 2 \end{pmatrix}$, $\det H = 8 - 4 = 4$, $H^{-1} = \frac{1}{4}\begin{pmatrix} 2 & 2 \\ 2 & 4 \end{pmatrix} = \begin{pmatrix} 0{,}5 & 0{,}5 \\ 0{,}5 & 1 \end{pmatrix}$.
2. (b) Redondeo: $\text{quant}(0{,}7) = 0{,}5$, $\text{quant}(0{,}3) = 0{,}5$; $\delta = (-0{,}2;\ 0{,}2)$; $\delta X = (-0{,}2 - 0{,}2;\ -0{,}2 + 0) = (-0{,}4;\ -0{,}2)$; $E = 0{,}16 + 0{,}04 = 0{,}20$.
3. (c) Se cuantiza w_1: $\varepsilon = 0{,}5 - 0{,}7 = -0{,}2$; $\delta = \varepsilon\,(H^{-1})_{1,:}/[H^{-1}]_{11} = -0{,}2 \cdot (0{,}5;\ 0{,}5)/0{,}5 = (-0{,}2;\ -0{,}2)$.
4. Pesos actualizados: $(0{,}5;\ 0{,}1)$, con error intermedio $\varepsilon^2/(2[H^{-1}]_{11}) = 0{,}04/1 = 0{,}04$.
5. Se cuantiza $w_2 = 0{,}1$: $\text{quant}(0{,}1) = 0$. Resultado $\hat{w} = (0{,}5;\ 0)$, $\delta = (-0{,}2;\ -0{,}3)$.
6. $\delta X = (-0{,}2 + 0{,}3;\ -0{,}2) = (0{,}1;\ -0{,}2)$; $E = 0{,}01 + 0{,}04 = 0{,}05$.

Resultado. $H^{-1} = \begin{pmatrix} 0{,}5 & 0{,}5 \\ 0{,}5 & 1 \end{pmatrix}$**; redondeo:** $\hat{w} = (0{,}5;\ 0{,}5)$**, error** $0{,}20$**; GPTQ:** $\hat{w} = (0{,}5;\ 0)$**, error** $0{,}05$**, cuatro veces menor.**

Comprobación. tras el primer paso, $\delta = (-0{,}2; -0{,}2)$ da $\delta X = (0; -0{,}2)$ y $E = 0{,}04$, que coincide con la fórmula del teorema 5.7. El peso w_2 se mueve en sentido contrario al redondeo ingenuo porque compensa el error de w_1 en la primera entrada.

Solución del ejercicio 5.5

Datos. $N = 6{,}5 \cdot 10^{10}$; fp16 (16 bits); NF4 (4 bits) con constantes de 0,5 o 0,127 bits por peso (proposición 5.10).

Planteamiento. memoria $= N \cdot$ (bits por peso)$/8$ bytes.

Desarrollo.

1. fp16: $6{,}5 \cdot 10^{10} \cdot 2 = 1{,}3 \cdot 10^{11}$ bytes $= 130$ GB.
2. NF4 con constantes fp32: 4,5 bits; $6{,}5 \cdot 10^{10} \cdot 4{,}5/8 = 3{,}656 \cdot 10^{10}$ bytes $= 36{,}56$ GB.
3. Doble cuantización: $4 + 0{,}126953 = 4{,}127$ bits; $6{,}5 \cdot 10^{10} \cdot 4{,}127/8 = 3{,}353 \cdot 10^{10}$ bytes $= 33{,}53$ GB.
4. Ahorro: $36{,}56 - 33{,}53 = 3{,}03$ GB, que es $6{,}5 \cdot 10^{10} \cdot 0{,}373/8$ bytes.

Resultado. 130 **GB en fp16,** 36,56 **GB en NF4 y** 33,53 **GB con doble cuantización; la doble cuantización ahorra unos** 3,03 **GB.**

Comprobación. $130/33{,}53 = 3{,}88$, algo menos que el factor $16/4 = 4$ por el coste de las constantes. Los 33 GB de pesos dejan margen en una GPU de 48 GB para los adaptadores y las activaciones.

Solución del ejercicio 5.6

Datos. $v = (3, 1, 0)$; $z = (2, 1, 1)$; $\tau \in \{1, 4\}$.

Planteamiento. $p = \mathrm{softmax}(v/\tau)$, $q = \mathrm{softmax}(z/\tau)$ y $\partial C/\partial z = (q - p)/\tau$ (proposición 5.12).

Desarrollo.

1. $\tau = 1$: $e^v = (20{,}09;\ 2{,}718;\ 1)$, suma 23,80, $p = (0{,}8438;\ 0{,}1142;\ 0{,}0420)$; $e^z = (7{,}389;\ 2{,}718;\ 2{,}718)$, suma 12,83, $q = (0{,}5761;\ 0{,}2119;\ 0{,}2119)$.
2. Gradiente con $\tau = 1$: $q - p = (-0{,}2677;\ 0{,}0977;\ 0{,}1699)$.
3. $\tau = 4$: $v/4 = (0{,}75;\ 0{,}25;\ 0)$, $e^{v/4} = (2{,}117;\ 1{,}284;\ 1)$, suma 4,401, $p = (0{,}4810;\ 0{,}2918;\ 0{,}2272)$; $z/4 = (0{,}5;\ 0{,}25;\ 0{,}25)$, $e^{z/4} = (1{,}649;\ 1{,}284;\ 1{,}284)$, suma 4,217, $q = (0{,}3910;\ 0{,}3045;\ 0{,}3045)$.
4. Gradiente con $\tau = 4$: $(q - p)/4 = (-0{,}0900;\ 0{,}0127;\ 0{,}0773)/4 = (-0{,}0225;\ 0{,}0032;\ 0{,}0193)$.
5. (c) $\tau^2 \cdot$ gradiente $= 16 \cdot (-0{,}0225;\ 0{,}0032;\ 0{,}0193) = (-0{,}360;\ 0{,}051;\ 0{,}309)$.

Resultado. $\tau = 1$**:** $p = (0{,}8438;\ 0{,}1142;\ 0{,}0420)$**,** $q = (0{,}5761;\ 0{,}2119;\ 0{,}2119)$**, gradiente** $(-0{,}2677;\ 0{,}0977;\ 0{,}1699)$**.** $\tau = 4$**:** $p = (0{,}4810;\ 0{,}2918;\ 0{,}2272)$**,** $q = (0{,}3910;\ 0{,}3045;\ 0{,}3045)$**, gradiente** $(-0{,}0225;\ 0{,}0032;\ 0{,}0193)$**.** Multiplicado por τ^2 recupera un orden de magnitud comparable al de $\tau = 1$.

Comprobación. cada gradiente suma 0, porque q y p suman 1: $-0{,}2677 + 0{,}0977 + 0{,}1699 = -0{,}0001 \approx 0$. Con $\tau = 4$ los objetivos blandos revelan que el profesor considera el segundo token bastante más probable que el tercero (0,29 frente a 0,23), información casi invisible con $\tau = 1$.

Solución del ejercicio 5.7

Datos. $\partial C/\partial z_i = (q_i - p_i)/\tau$; $|z_i|/\tau$ y $|v_i|/\tau$ pequeños; $\sum_i z_i = \sum_i v_i = 0$.

Planteamiento. linealizar las exponenciales de las dos softmax [10, §2.1].

Desarrollo.

1. $q_i = \dfrac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}} \approx \dfrac{1 + z_i/\tau}{K + \sum_j z_j/\tau} = \dfrac{1 + z_i/\tau}{K}$, por la media nula; análogamente $p_i \approx (1 + v_i/\tau)/K$.
2. $\dfrac{\partial C}{\partial z_i} \approx \dfrac{1}{\tau} \cdot \dfrac{z_i - v_i}{K\tau} = \dfrac{z_i - v_i}{K\tau^2}$. ■
3. (b) La función $G(z) = \frac{1}{2K\tau^2} \sum_i (z_i - v_i)^2$ tiene gradiente $\partial G/\partial z_i = (z_i - v_i)/(K\tau^2)$, el mismo; minimizar C equivale, en ese régimen, a igualar logits por mínimos cuadrados.
4. (c) El gradiente blando es proporcional a $1/\tau^2$; si se combina con la pérdida de las etiquetas (a temperatura 1) sin corregir, su peso relativo cambiaría con τ. Multiplicar por τ^2 lo mantiene aproximadamente constante.
5. (d) Centrado: $v - \bar{v} = (5/3; -1/3; -4/3)$, $z - \bar{z} = (2/3; -1/3; -1/3)$; diferencia $(-1; 0; 1)$. Aproximación con $K = 3$, $\tau = 10$: $(-1; 0; 1)/300 = (-0{,}00333; 0; 0{,}00333)$.
6. Valor exacto con $\tau = 10$ (la softmax no cambia al centrar): $p = (0{,}3907; 0{,}3199; 0{,}2894)$, $q = (0{,}3559; 0{,}3220; 0{,}3220)$, $(q - p)/10 = (-0{,}00348; 0{,}00022; 0{,}00326)$.

Resultado. $\partial C/\partial z_i \approx (z_i - v_i)/(K\tau^2)$**; la destilación a temperatura alta iguala logits por mínimos cuadrados, y el factor** τ^2 **compensa la escala** $1/\tau^2$**.** Con $\tau = 10$ la aproximación $(-0{,}00333; 0; 0{,}00333)$ se acerca al valor exacto $(-0{,}00348; 0{,}00022; 0{,}00326)$.

Comprobación. con $\tau = 4$ (ejercicio 5.6) la aproximación da $(-1; 0; 1)/48 = (-0{,}0208; 0; 0{,}0208)$ frente a $(-0{,}0225; 0{,}0032; 0{,}0193)$: el error relativo disminuye al crecer τ.

A.6 Tema 6

Solución del ejercicio 6.1

Planteamiento. comparar la definición 6.8 con la memoria virtual paginada.

Desarrollo.

1. Un proceso ve un espacio de direcciones contiguo formado por páginas físicas dispersas; la tabla de páginas traduce direcciones lógicas a físicas. En PagedAttention, cada secuencia ve una caché lógica contigua formada por bloques físicos dispersos, y la tabla de bloques hace la traducción.
2. Los bloques se asignan cuando se necesitan, de modo que no hay que reservar el tamaño máximo: desaparece casi toda la fragmentación interna (solo queda parte del último bloque, proposición 6.9). Como todos los bloques son iguales, no hay fragmentación externa.
3. Compartir páginas de solo lectura entre procesos equivale a que varias secuencias apunten a los mismos bloques del prompt común; la copia en escritura de los sistemas operativos corresponde a copiar el bloque parcial cuando una secuencia escribe en él (proposición 6.11).

Resultado. **PagedAttention elimina la reserva contigua y la fragmentación, y permite compartir bloques con copia en escritura, igual que la paginación de un sistema operativo.**

Solución del ejercicio 6.2

Datos. $\ell = 1000$; $b_{\text{blk}} = 16$; reserva contigua de 2048.

Planteamiento. proposición 6.9.

Desarrollo.

1. $1000/16 = 62{,}5$, luego $\lceil 62{,}5 \rceil = 63$ bloques.
2. Posiciones: $63 \cdot 16 = 1008$; desperdicio $1008 - 1000 = 8$, es decir, $8/1008 = 0{,}794\,\%$.
3. Reserva contigua: $2048 - 1000 = 1048$ posiciones vacías, $1048/2048 = 51{,}17\,\%$.

Resultado. **63 bloques y 8 posiciones desperdiciadas** ($0{,}794\,\%$)**, frente a 1048** ($51{,}17\,\%$) **con la reserva contigua.**

Comprobación. $8 \leq b_{\text{blk}} - 1 = 15$, como garantiza la proposición 6.9.

Solución del ejercicio 6.3

Datos. $N = 1{,}3 \cdot 10^{10}$; fp16; $1{,}5 \cdot 10^{14}$ FLOP/s efectivos en el *prefill*; $B_{\text{mem}} = 10^{12}$ bytes/s; $n = 2000$; $n_{\text{out}} = 500$; longitudes 100, 400, 250, 50.

Planteamiento. TTFT $\approx 2Nn/$(FLOP/s efectivos) (ejemplo 2.2); TPOT $\approx sN/B_{\text{mem}}$ (teorema 2.10); latencia por la proposición 6.2; utilización por la proposición 6.5.

Desarrollo.

1. TTFT: $2 \cdot 1{,}3 \cdot 10^{10} \cdot 2000 = 5{,}2 \cdot 10^{13}$ FLOP; $5{,}2 \cdot 10^{13}/1{,}5 \cdot 10^{14} = 0{,}3467$ s.
2. TPOT: $2{,}6 \cdot 10^{10}/10^{12} = 0{,}026$ s (38,46 tokens/s).
3. Latencia: $0{,}3467 + 499 \cdot 0{,}026 = 0{,}3467 + 12{,}97 = 13{,}32$ s.
4. (b) $\sum \ell_i = 800$; $B \max \ell_i = 4 \cdot 400 = 1600$; $U = 800/1600 = 0{,}5$.

Resultado. **(a)** TTFT $\approx 0{,}347$ **s,** TPOT ≈ 26 **ms y latencia total** $\approx 13{,}32$ **s. (b) Utilización del** $50\,\%$**.**

Comprobación. la decodificación domina la latencia (12,97 de 13,32 s) aunque el *prefill* procese cuatro veces más tokens, porque cada paso de decodificación está limitado por la lectura de los 26 GB de pesos.

Solución del ejercicio 6.4

Datos. $N' = 1{,}1338 \cdot 10^{10}$; bf16; $L = 40$, $n_{\text{kv}} = 8$, $d_{\text{h}} = 128$; $S = 1024$; $B_{\text{mem}} = 3{,}35 \cdot 10^{12}$; 80 GB.

Planteamiento. los mismos cálculos de la solución del ejercicio 2.5 con $n_{\text{kv}} = 8$.

Desarrollo.

1. $c_{\text{kv}} = 2 \cdot 40 \cdot 8 \cdot 128 \cdot 2 = 163\,840$ bytes; por secuencia, $1024 \cdot 163\,840 = 1{,}678 \cdot 10^{8}$ bytes $= 0{,}1678$ GB, cinco veces menos.
2. Pesos: $2 \cdot 1{,}1338 \cdot 10^{10} = 22{,}68$ GB.

3. $B = 64$: caché 10,74 GB; memoria 33,41 GB; latencia $33{,}41 \cdot 10^9 / 3{,}35 \cdot 10^{12} = 9{,}97$ ms; rendimiento $64/0{,}00997 = 6417$ tokens/s.
4. $B = 256$: caché 42,95 GB; memoria 65,63 GB; latencia 19,59 ms; rendimiento $256/0{,}01959 = 13\,068$ tokens/s.
5. Ambos caben en 80 GB. Con atención multicabeza, $B = 64$ daba 23,80 ms y 2689 tokens/s, y $B = 256$ no cabía.

Resultado. **caché de** 0,168 **GB por secuencia;** $B = 64$**:** 33,41 **GB,** 9,97 **ms,** 6417 **tokens/s;** $B = 256$**:** 65,63 **GB,** 19,59 **ms,** 13 068 **tokens/s.** GQA permite lotes cuatro veces mayores en la misma GPU y multiplica el rendimiento por casi cinco respecto al mejor caso que cabía con atención multicabeza.

Comprobación. la caché se divide exactamente por $40/8 = 5$: $0{,}8389/0{,}1678 = 5{,}0$.

Solución del ejercicio 6.5

Datos. $n_s = 4$; $P = 1000$; $b_{\text{blk}} = 16$; 512 KiB por token.

Planteamiento. proposición 6.11.

Desarrollo.

1. Sin compartir: cada muestra necesita $\lceil 1000/16 \rceil = 63$ bloques; en total $4 \cdot 63 = 252$.
2. Con compartición: $\lfloor 1000/16 \rfloor = 62$ bloques completos compartidos; el bloque parcial ($1000 - 62 \cdot 16 = 8$ tokens) se copia una vez por muestra: $62 + 4 = 66$ bloques.
3. Ahorro: $252 - 66 = 186$ bloques. Cada bloque ocupa $16 \cdot 512 = 8192$ KiB $= 8$ MiB, luego se ahorran $186 \cdot 8 = 1488$ MiB.

Resultado. **252 bloques sin compartir frente a 66 con compartición; se ahorran 186 bloques, unos 1488 MiB (**1,45 **GiB).**

Comprobación. el ahorro es $(n_s - 1)\lfloor P/b_{\text{blk}} \rfloor = 3 \cdot 62 = 186$ bloques: cada muestra adicional reutiliza los 62 bloques completos.

Solución del ejercicio 6.6

Datos. $r = \ell \bmod b_{\text{blk}}$ uniforme en $\{0, \dots, b_{\text{blk}} - 1\}$; en (b), ℓ uniforme en $\{1, \dots, L_{\max}\}$.

Planteamiento. expresar el desperdicio en función de r o de ℓ y tomar esperanzas.

Desarrollo.

1. (a) Por la proposición 6.9, el desperdicio es $w = b_{\text{blk}}\lceil \ell / b_{\text{blk}} \rceil - \ell$, que vale 0 si $r = 0$ y $b_{\text{blk}} - r$ si $r \in \{1, \dots, b_{\text{blk}} - 1\}$.
2. Cuando r recorre $\{0, \dots, b_{\text{blk}} - 1\}$, w recorre $\{0\} \cup \{b_{\text{blk}} - 1, \dots, 1\} = \{0, \dots, b_{\text{blk}} - 1\}$, cada valor una vez; luego w es uniforme en ese conjunto.
3. $\mathbb{E}[w] = \frac{1}{b_{\text{blk}}} \sum_{j=0}^{b_{\text{blk}}-1} j = \frac{1}{b_{\text{blk}}} \cdot \frac{(b_{\text{blk}}-1)b_{\text{blk}}}{2} = \frac{b_{\text{blk}}-1}{2}$. ■
4. (b) Con reserva contigua el desperdicio es $L_{\max} - \ell$ y $\mathbb{E}[\ell] = (L_{\max}+1)/2$, luego $\mathbb{E}[L_{\max} - \ell] = (L_{\max} - 1)/2$. ■
5. (c) $(16 - 1)/2 = 7{,}5$ posiciones; $(2048 - 1)/2 = 1023{,}5$ posiciones.

Resultado. **desperdicio esperado de** $(b_{\mathrm{blk}} - 1)/2 = 7{,}5$ **posiciones con bloques de 16, frente a** $(L_{\max} - 1)/2 = 1023{,}5$ **con reserva contigua de 2048: unas 136 veces menos.**

Comprobación. 1023,5/7,5 = 136,5. El modelo de (b) predice un uso medio del 50 % de la reserva; Kwon *et al.* miden entre el 20,4 % y el 38,2 % en sistemas reales, que además reservan para el caso peor y sufren fragmentación externa [14].

Bibliografía

[1] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre y J. Jumper. Accelerating large language model decoding with speculative sampling. Prepublicación, DeepMind, 2023. https://arxiv.org/abs/2302.01318

[2] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. Sección 6, «Generating text». https://github.com/stanford-cs336/assignment1-basics

[3] T. Dettmers, M. Lewis, Y. Belkada y L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for Transformers at scale. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2208.07339

[4] T. Dettmers, A. Pagnoni, A. Holtzman y L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.14314

[5] A. Fan, M. Lewis e Y. Dauphin. Hierarchical neural story generation. En *Proceedings of ACL*, 2018. https://arxiv.org/abs/1805.04833

[6] E. Frantar, S. Ashkboos, T. Hoefler y D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained Transformers. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2210.17323

[7] R. M. Gray y D. L. Neuhoff. Quantization. *IEEE Transactions on Information Theory*, 44(6):2325–2383, 1998.

[8] S. Han. 6.5940: TinyML and Efficient Deep Learning Computing. Massachusetts Institute of Technology, otoño de 2024. Clases 5 y 6 («Quantization»), 9 («Knowledge Distillation») y 13 («Efficient LLM Deployment»). https://hanlab.mit.edu/courses/2024-fall-65940

[9] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Clase 10, «Inference». https://github.com/stanford-cs336/lectures

[10] G. Hinton, O. Vinyals y J. Dean. Distilling the knowledge in a neural network. Prepublicación, 2015 (*NIPS Deep Learning Workshop*, 2014). https://arxiv.org/abs/1503.02531

[11] A. Holtzman, J. Buys, L. Du, M. Forbes e Y. Choi. The curious case of neural text degeneration. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1904.09751

[12] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam y D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. https://arxiv.org/abs/1712.05877

[13] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining» (§7.6 «Decoding»), y cap. 13, «Machine Translation» (§13.4 «Decoding in MT: Beam Search»). https://web.stanford.edu/~jurafsky/slp3/

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang e I. Stoica. Efficient memory management for large language model serving with PagedAttention. En *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. https://arxiv.org/abs/2309.06180

[15] Y. Leviathan, M. Kalman e Y. Matias. Fast inference from Transformers via speculative decoding. En *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 2023. https://arxiv.org/abs/2211.17192

[16] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan y S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. En *Proceedings of Machine Learning and Systems (MLSys)*, 2024. https://arxiv.org/abs/2306.00978

[17] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal y J. Dean. Efficiently scaling Transformer inference. En *Proceedings of Machine Learning and Systems (MLSys)*, 2023. https://arxiv.org/abs/2211.05102

[18] N. Shazeer. Fast Transformer decoding: One write-head is all you need. Prepublicación, 2019. https://arxiv.org/abs/1911.02150

[19] University of Washington. CSE 493G1/599G1: Deep Learning for Computer Vision. Primavera de 2025. Sección (*recitation*) 4, «Quantization». https://courses.cs.washington.edu/courses/cse493g1/25sp/schedule/recitations/rec4.pdf

[20] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth y S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. En *Proceedings of the 40th International Conference on Machine Learning (ICML)*, PMLR 202, 2023. https://arxiv.org/abs/2211.10438

[21] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim y B.-G. Chun. Orca: A distributed serving system for Transformer-based generative models. En *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. https://www.usenix.org/system/files/osdi22-yu.pdf

[22] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §4.1 «Softmax Regression» y §10.8 «Beam Search». https://d2l.ai/